



Introduction to Blockchain

CSE-355

Ahmed Akhtar

Department of Computer Science

SMCS, IBA Karachi

Spring 2026

Lecture 21

Tuesday, Apr 14, 2026

Risks of Turing-Completeness

Ethereum is Turing-complete, which allows immense flexibility in writing smart contracts

Are there any challenges and risks of Turing-Completeness?

There are several risks:

- **Security risks:** Due to complexities and vulnerabilities that are difficult to anticipate
- **Difficulty in formal verification:** Proving or verifying the correctness of a program becomes challenging
- **Scalability concerns:** Arbitrary and computationally resource-intensive programs can be created
- **Potential for misuse or abuse:** The flexibility of Turing-complete languages can also be exploited for malicious purposes, e.g., malicious actors may create smart contracts to conceal malicious behavior, making it harder to detect and mitigate security risks
- Infinite loops that never terminate can be easily created
- Unintended never-ending loops can also arise without warning

Bitcoin's scripting language is intentionally Turing-incomplete to ensure predictable outcomes

Introduce Computational Cost: Gas

To address the disadvantages of Turing-completeness, Ethereum introduced a computational cost mechanism called '*Gas*'

- Every transaction that requires computational effort must include sufficient gas
- Instructions such as computations and data access have predetermined computational costs measured in gas units
- Computation terminates prematurely if the gas limit is exhausted and there are still some instructions pending
 - In such cases, the transaction fails and gas spent is lost
- Ethereum does not require users to provide the exact gas amount for their smart contracts
 - Users may include a higher amount, as any unused gas is returned if the transactions executes successfully

The use of gas as the fundamental unit of computational effort allows Ethereum to limit resource consumption and prevent misuse by malicious actors

Gas Cost of Instructions

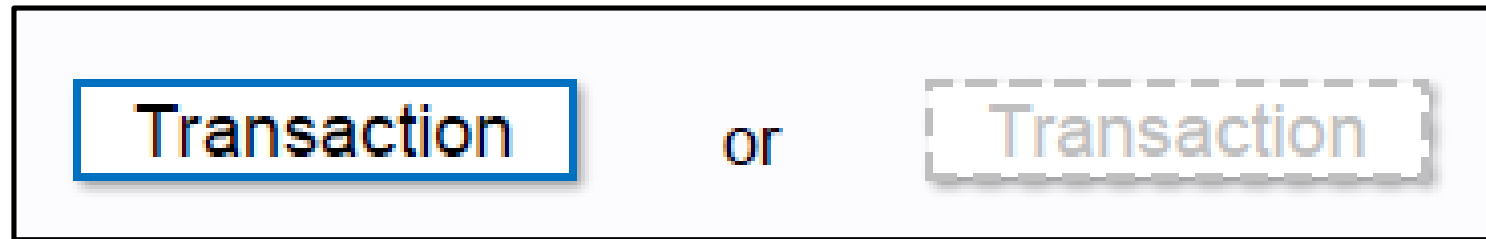
Value	Mnemonic	Gas Used	Notes
0x00	STOP	0	Halts execution
0x01	ADD	3	Addition operation
0x02	MUL	5	Multiplication operation
0x03	SUB	3	Subtraction operation
0x04	DIV	5	Integer division operation
0x05	SDIV	5	Signed integer division operation (truncated)
0x06	MOD	5	Modulo remainder operation
0x07	SMOD	5	Signed modulo remainder operation
0x08	ADDMOD	8	Modulo addition operation
0x09	MULMOD	8	Modulo multiplication operation

Value	Mnemonic	Gas Used	Notes
0x31	BALANCE	400	Get balance of the given account
0x32	ORIGIN	2	Get execution origination address
0x33	CALLER	2	Get caller address
0x3b	EXTCODESIZE	700	Get size of an account's code
0x54	SLOAD	200	Load word from storage
0xf0	CREATE	53,000	Create a new account with code (smart contract deployment)
0x37	CALLDATACOPY	FORMULA	Copy input data in current environment to memory
0x38	CODESIZE	2	Get size of code running in current environment
0x39	CODECOPY	FORMULA	Copy code running in current environment to memory

https://docs.google.com/spreadsheets/d/1n6mRqkBz3iWcOIRem_mO0gGtSKEKrAsfO7Frgx18pNU/edit#gid=0

Transactions are Atomic

A transaction is treated as an atomic operation that cannot be divided or interrupted



All (completely done) or Nothing (zero effect)

If an insufficient amount of gas is included in a transaction, all the state changes are rolled back except for the gas, which gets consumed while performing the computational steps

Introduction of gas to measure computational effort and treatment of transaction as atomic operation are Ethereum's innovations to overcome the disadvantages of Turing-completeness

Attack Scenario: Example

Attack Scenario: An attacker creates a contract which runs an infinite loop and then sends a transaction activating that loop to the miner/validator. The miner/validator does not know about the infinite loop and starts processing the transaction

- Will this attack succeed?
 - No
- Why?
 - A valid txn must include gas limit, therefore, the execution will stop when this limit is reached
- Is this txn included in the block?
 - Yes, it is included as a failed txn
 - The fee for the computational effort goes to the miner/validator

Failed Transaction: Example

<https://etherscan.io/>

Transaction Hash:	0x3fa19306a5dfc227537fdaf90b6883104dd0458cf9fbe3410a3254e5ca0e3618
Status:	❌ Fail
Block:	12910372 1282307 Block Confirmations
Timestamp:	199 days 20 hrs ago (Jul-27-2021 09:06:35 PM +UTC) Confirmed within 30 secs
From:	superphiz.eth
To:	Contract 0xd4d871419714b778ebec2e22c7c53572b573706e (Stoner Cats: TOKEN Token) ⚠️
	Warning! Error encountered during contract execution [Out of gas]
Value:	7 Ether (\$20,485.92) - [CANCELLED]
Transaction Fee:	5.818454 Ether (\$17,028.05) ← Fee paid by the user for this failed txn
Gas Price:	0.000002 Ether (2,000 Gwei)
Ether Price:	\$2,301.98 / ETH

<https://etherscan.io/tx/0x3fa19306a5dfc227537fdaf90b6883104dd0458cf9fbe3410a3254e5ca0e3618>

Attack Scenario: Example

Attack Scenario: An attacker creates a very long loop with the intent of forcing the miner/validator to keep computing for such a long time that by the time computation finishes a few more blocks will have come out and it will not be possible for the miner/validator to include the transaction to claim the fee

- Will this attack succeed?
 - No
- Why?
 - A valid txn includes gas limit that will allow the miner/validator to estimate the computation time before deciding to process it

Attack Scenario: Example

Attack Scenario: Suppose there is a contract on blockchain that allows withdrawal from an account followed by updating the account balance. An attacker who can withdraw amount creates a txn to execute this contract but includes just enough gas to run the first step i.e., making a withdrawal but not letting his balance go down

- Will this attack succeed?
 - No
- Why?
 - Transactions are treated as atomic operations. If this execution stops halfway through, the changes will be reverted, and withdrawal of funds request will be cancelled

Gas is Purchased with Ether

- Gas is an essential requirement for anyone to use Ethereum
- Gas can only be purchased with Ether (the cryptocurrency of Ethereum)
- Ether is identified on crypto exchanges as 'ETH'

Purchasing gas for various computations (txns and smart contracts) is one of the major use of Ethereum's cryptocurrency (Ether)

Purchase gas for
payment of txn fees

Digital store of value
like Bitcoin

Decentralized Finance
(crypto loans/staking)

The smallest denomination, base unit, of ether is called Wei:

$$1\text{ETH} = 10^{18} \text{ wei}$$

- Gas price is generally specified in Gwei
- The number of gas units required for specific computations always remain the same, but the gas price varies

ETH Supply Model

When Ethereum launched in 2015, Vitalik Buterin chose not to cap the supply of Ether

The idea was to follow a linear supply model:

$$(15.6 \times n + 72) \text{ Million}$$

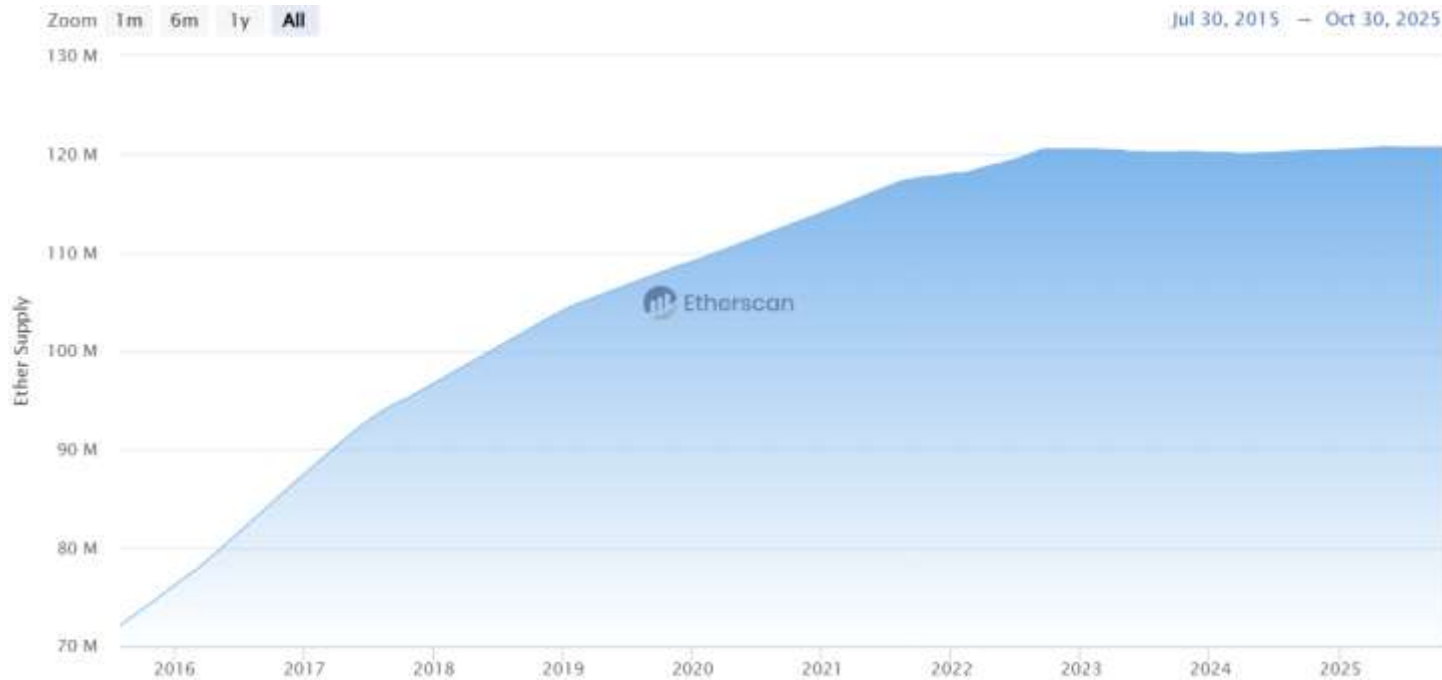
- n denotes the number of years since the genesis block (July 30, 2015)
- 72 M ETH were created at the launch
 - 60 M were sold to public during the presale at a rate of 2000 ETH per BTC
 - 12 M were allocated to the Ethereum foundation and early developers

Assuming $n = 10$ years since genesis, the total Ether supply would be approximately 228 million

Current ETH supply is only:
 $120.7 \text{ M} < 228 \text{ M}$

- Mining reward was gradually reduced from 5 ETH to 2 ETH, and after the Merge, it became 0 ETH (no more miners)
- New ETH is now generated only to pay staking rewards to validators
- Ether burning, which permanently removes some ETH from circulation, began in August 2021 after the London upgrade

ETH Supply & ETH Burning



The ETH supply has stabilized around 120 M due to EIP-1559 (London upgrade, Aug 2021) and the Merge (Sept 2022), which reduced ETH issuance and introduced ETH burning

March 21, 2023



March 28, 2024



Oct 31, 2025

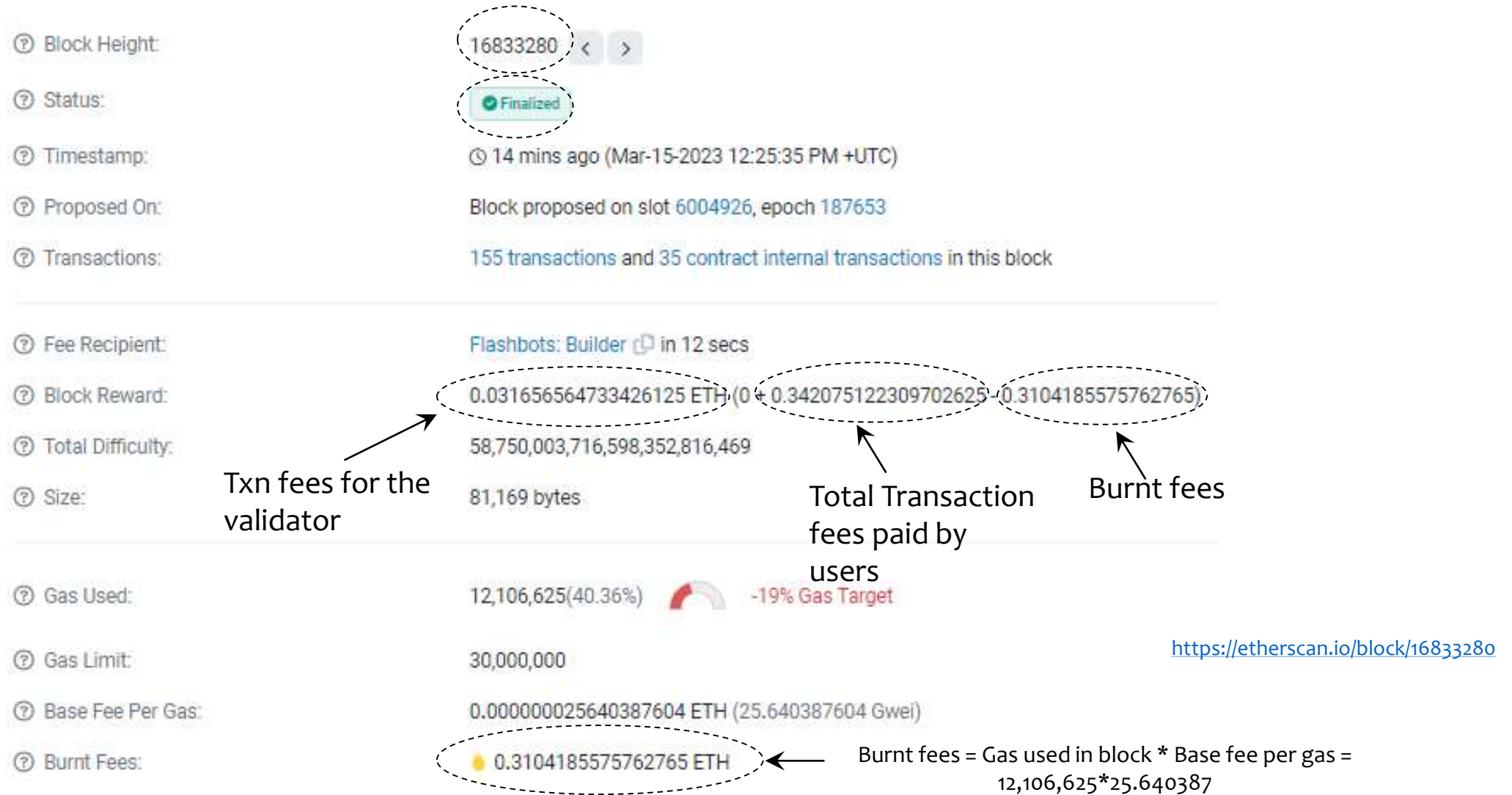


More than 4.6 million
ETH has been burned
till now

<https://etherscan.io/chart/ethersupplygrowth>

<https://etherscan.io/chart/dailyethburnt>

ETH Burning



Price vs Market Cap



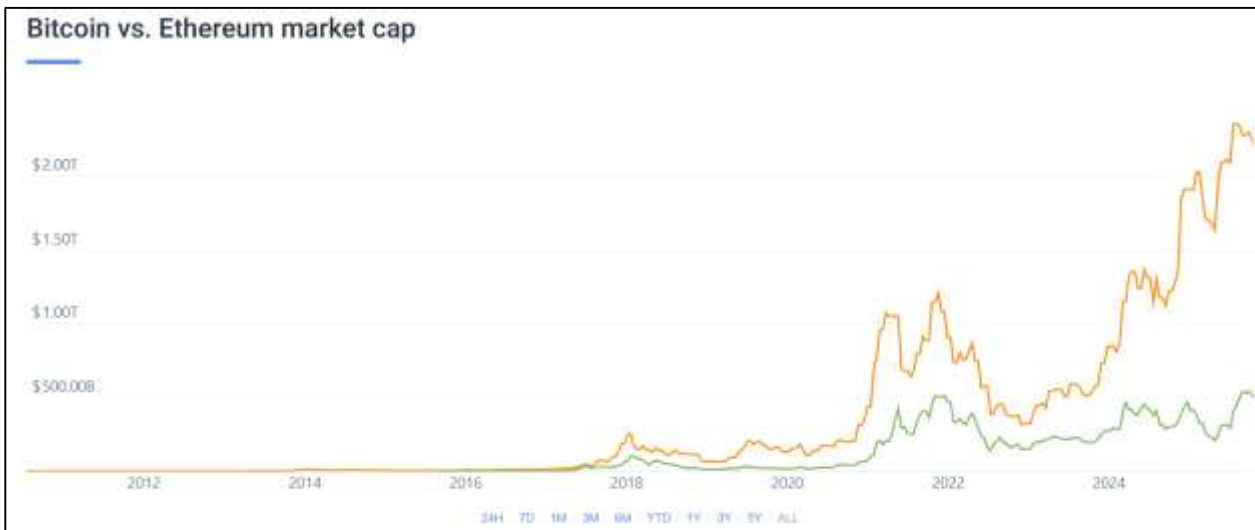
- ETH price = \$3.8k, ETH current supply = 120.7 M
- BTC price = \$110k, BTC current supply = 19.7 M

• Market Capitalization (cap): Price x Supply

- ETH market cap: \$458B (~ \$500B)
- BTC market cap: \$2.16T

• Bitcoin is roughly four times as valuable as Ethereum

- Market cap provides a more complete picture of a network's overall worth
- Bitcoin market cap has increased more rapidly in recent years compared to ETH



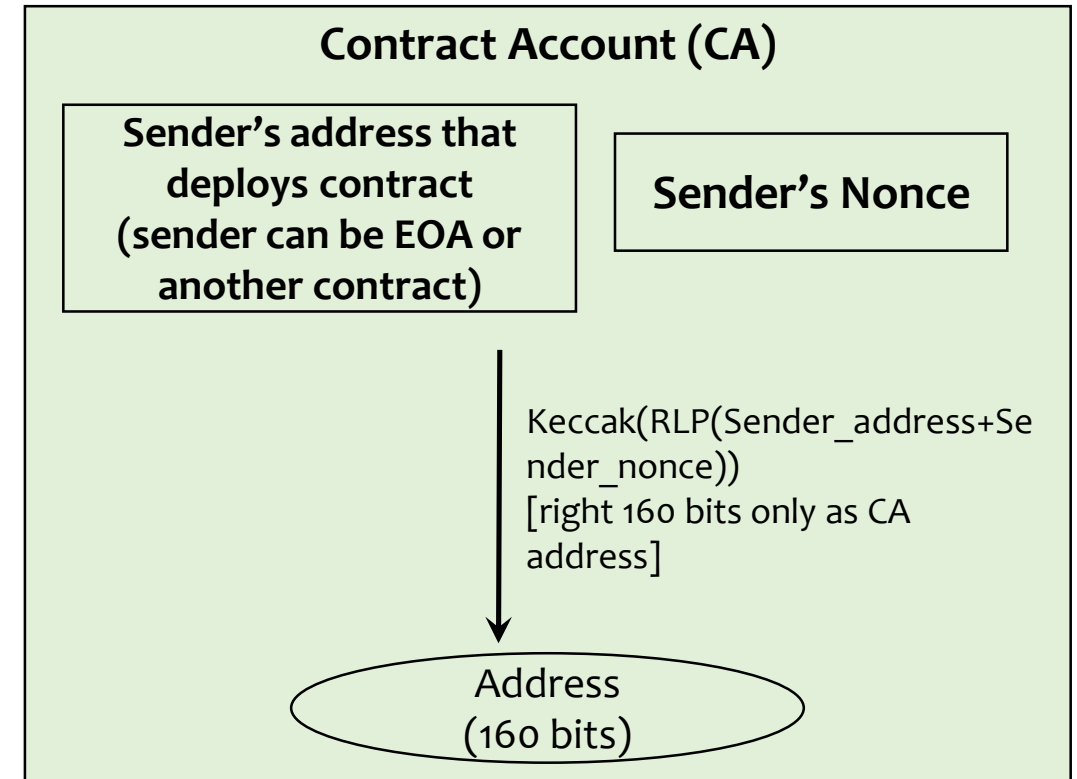
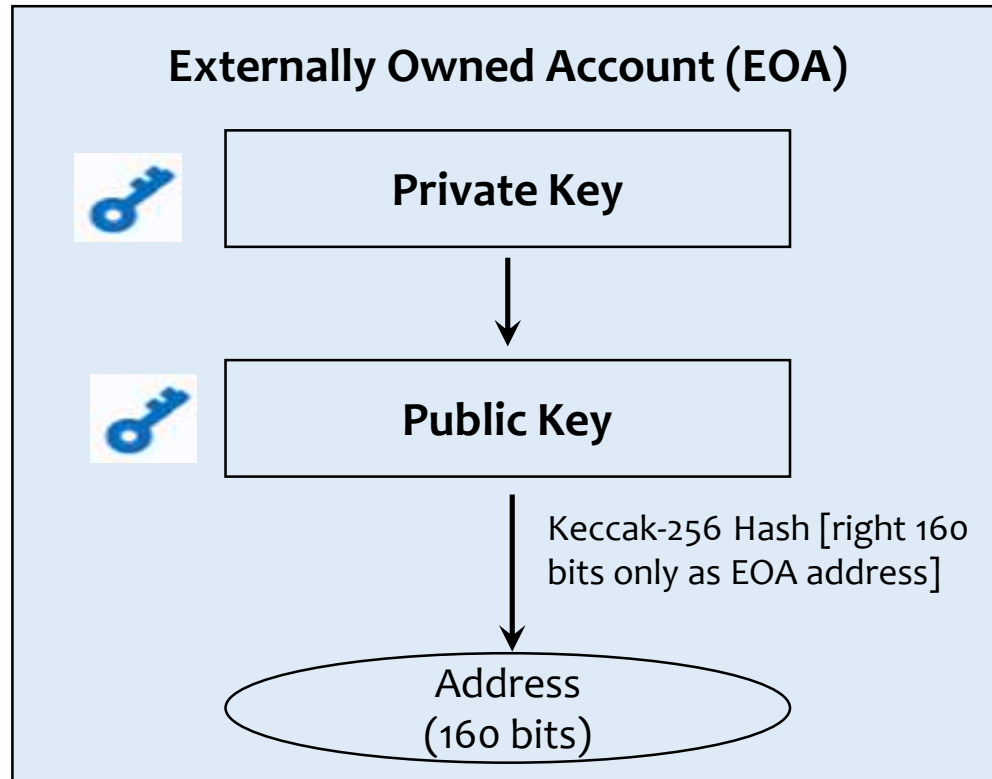
Lecture 22

Thursday, Apr 16, 2026

Ethereum: Technical Details

Account Types in Ethereum

- Ethereum uses account based model and there are no UTXOs like Bitcoin
- There are two types of Ethereum accounts



RLP (recursive length prefix) is an encoding or serialization technique while Keccak-256 is the hashing algorithm used by Ethereum

Externally Owned Account (EOA)

A user or a wallet can create an EOA: Creating an EOA costs nothing

1 Keypair = 1 EOA

- EOA is made up of a cryptographic key pair: public and private
- First a private key (32B) is generated whose purpose is to sign the transactions (like Bitcoin)
 - it is randomly generated from a seed
- A public key (64B) is derived from the private key using Elliptic curve digital signature algorithm (ECDSA)
- Account address (20B) is the last 160 bits of the Keccak-256 hash of the public key

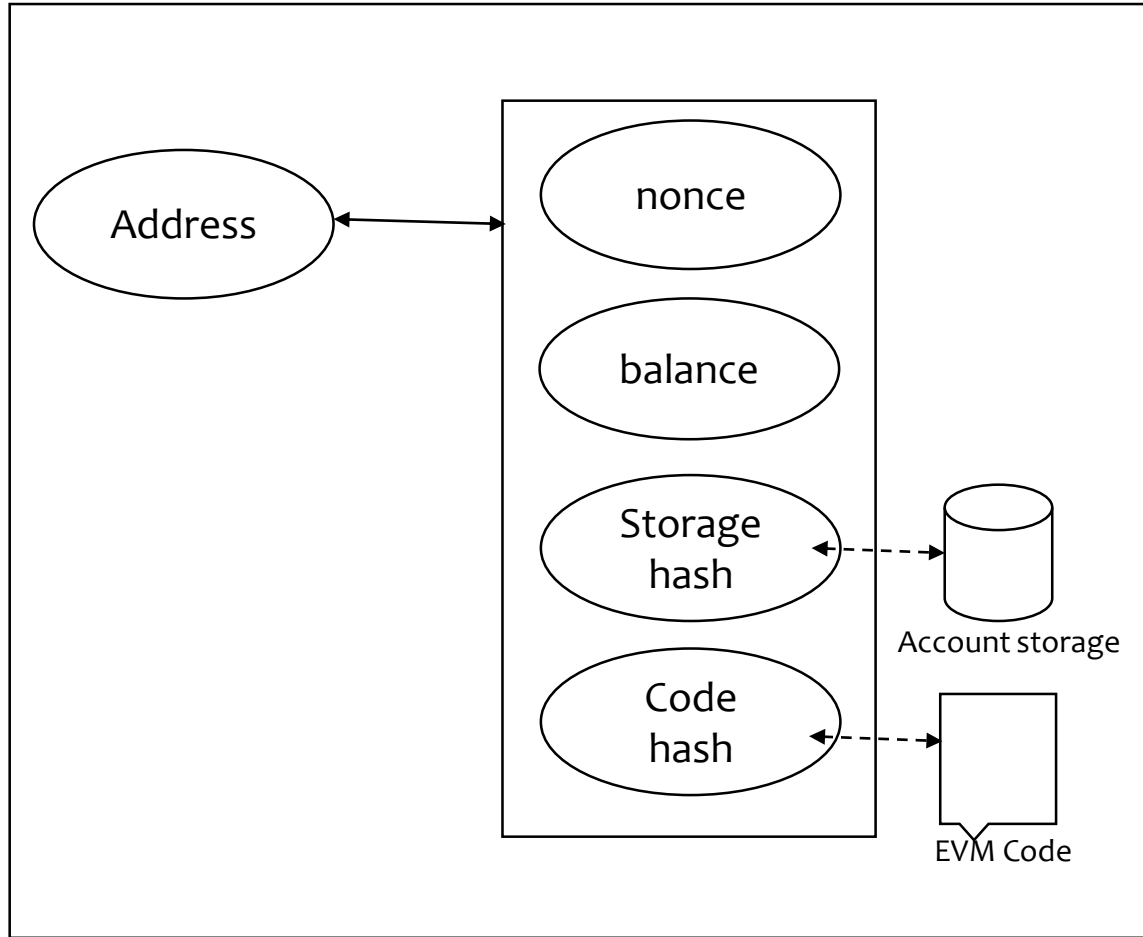
- Can initiate txns
- Receive, hold and send ETH
- Interact with txns and deployed smart contracts
- Txns between EOAs can only be ETH/token transfers

Contract Account (CA)

- CA is the address of a smart contract deployed to the network
 - It is controlled by its code
 - Creating a contract account has cost in terms of gas
- Contract address (20B) becomes available after its deployment
 - It comes from contract creator's address (either EOA or another smart contract) and number of transactions sent from that address (nonce)
- *CA cannot initiate transactions*
- CA can interact with txns and other smart contracts
- CA can receive, hold and send ETH or tokens
- CA can create and deploy new smart contracts e.g., factory contract or smart contract factory

Only EOAs can start a transaction

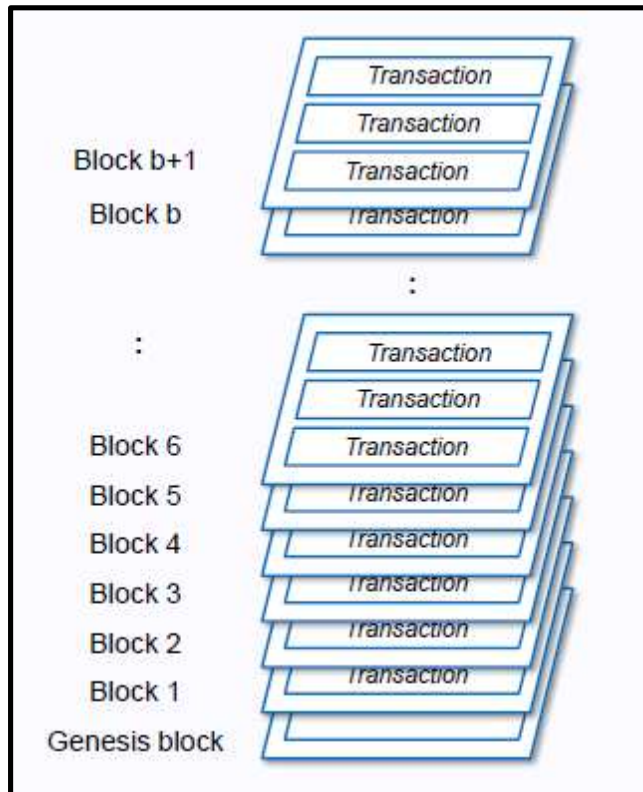
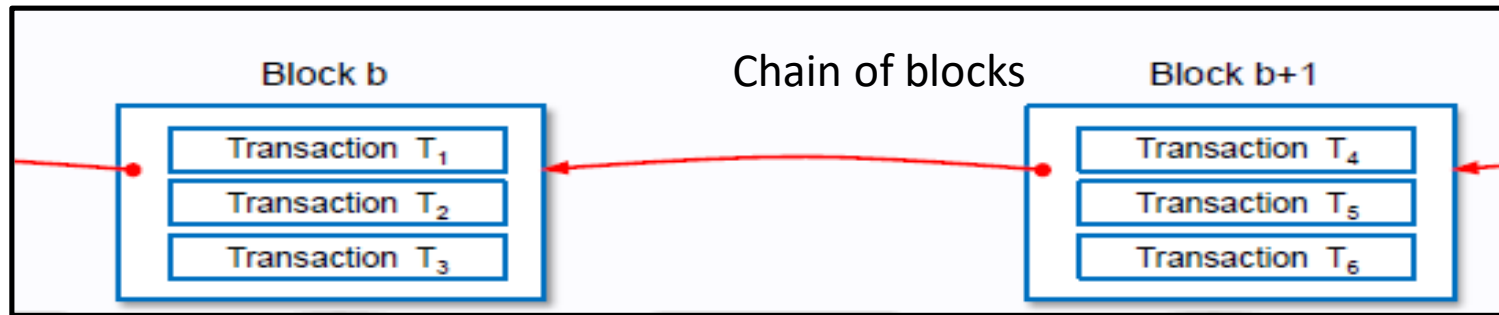
Account State



- Nonce is a counter that is incremented every time a txn is made by an account
 - It is used to protect against replay attacks
 - EOA Nonce is number of transactions performed by the account
 - CA Nonce is the number of contracts created by the account
- Storage Hash: The storage mechanism provides a verifiable summary of a contract's stored data (e.g., token balances)
 - A Merkel like tree is constructed and a 256-bit hash of its root node is stored as part of the account state
- Code Hash: This refers to the hash of the bytecode associated with an account
 - EOA: hash of an empty string
 - CA: hash of the smart contract code

<https://ethereum.org/en/developers/docs/accounts/>

Blockchain



In case of Bitcoin, transactions are stored in blocks and all the required information is included in a Bitcoin txn

Prev Txid: A1234A Idx: 0 scriptSig <Alice>	Value: 0.01 BTC scriptPubKey = <Data>
	Value: 1.49 BTC scriptPubKey <Alice>

Bitcoin Transaction

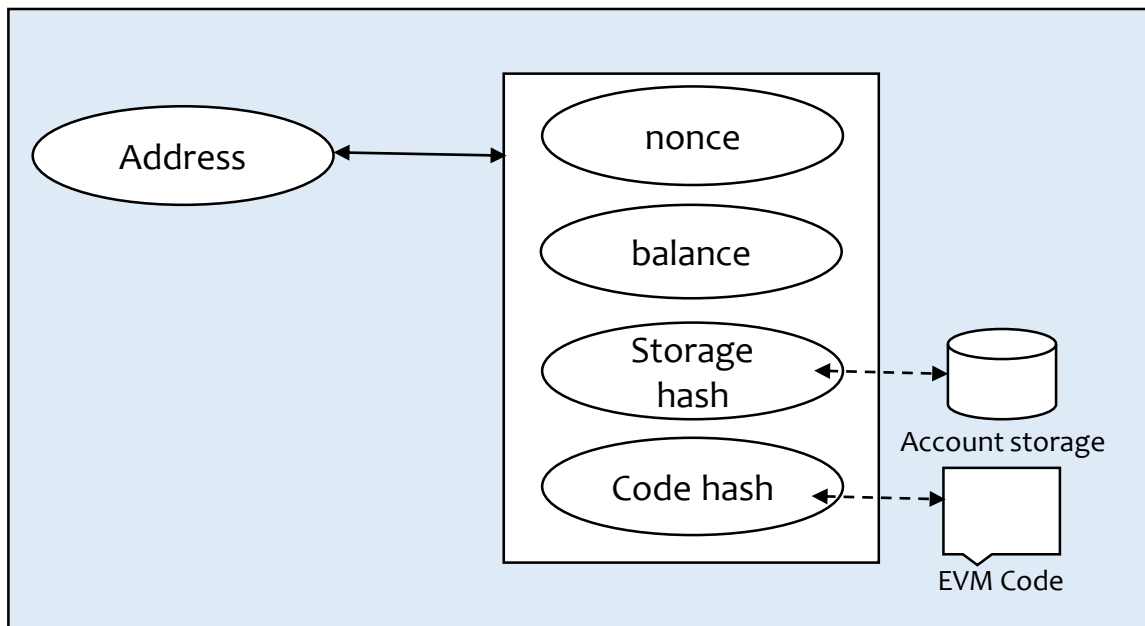
What about Ethereum? Do we store txns in blocks? What's inside a txn? Account information, code, data, something else?

Alternate View of Blockchain – World State

Due to account based model, we adopt a different approach to understand Ethereum

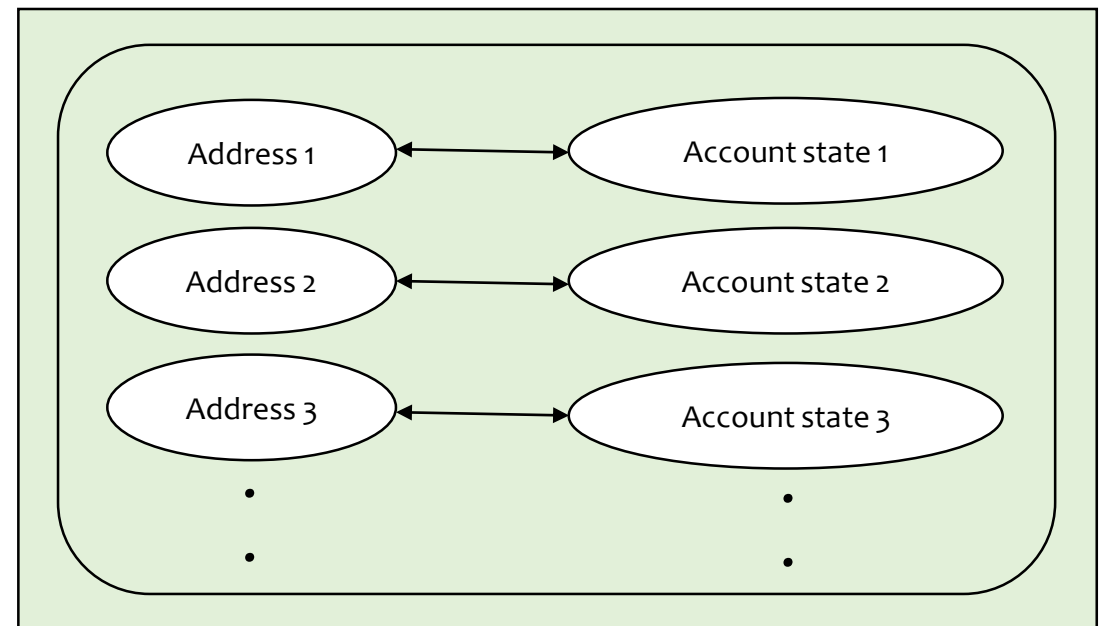
Every Ethereum account has a state that is represented by a nonce, balance, codehash and storagehash

Single Account State



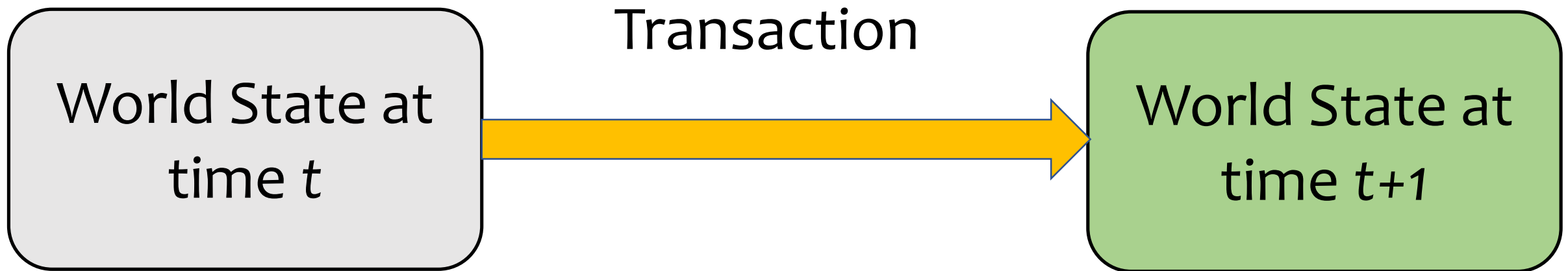
The world state (simply state) of Ethereum at any time t is defined as the state of all the accounts

World State



Transaction

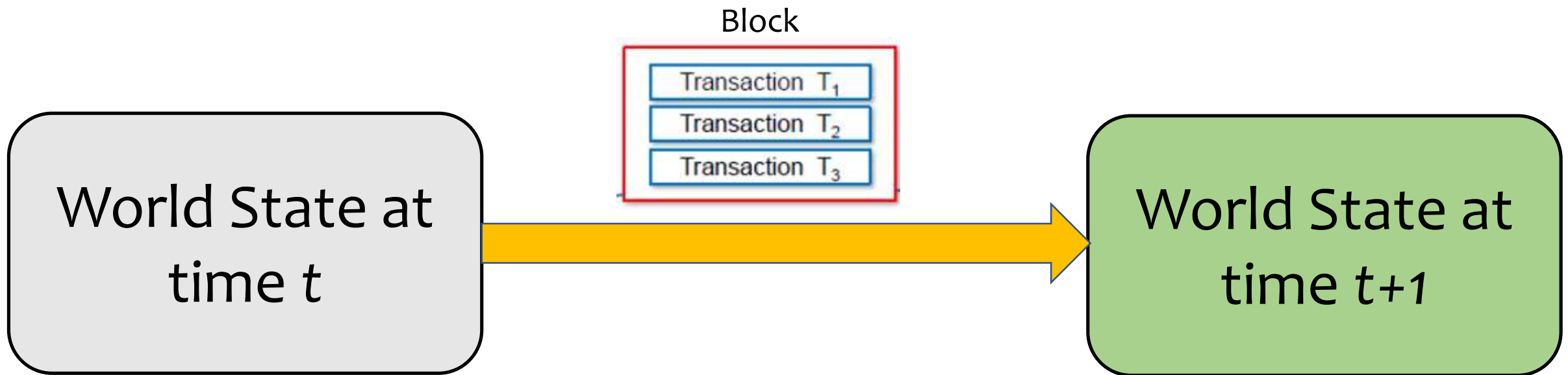
A transactions is a single cryptographically signed instruction initiated by an EOA that changes Ethereum world state



Only valid transactions change the world state because invalid txns are rejected by the network

Block

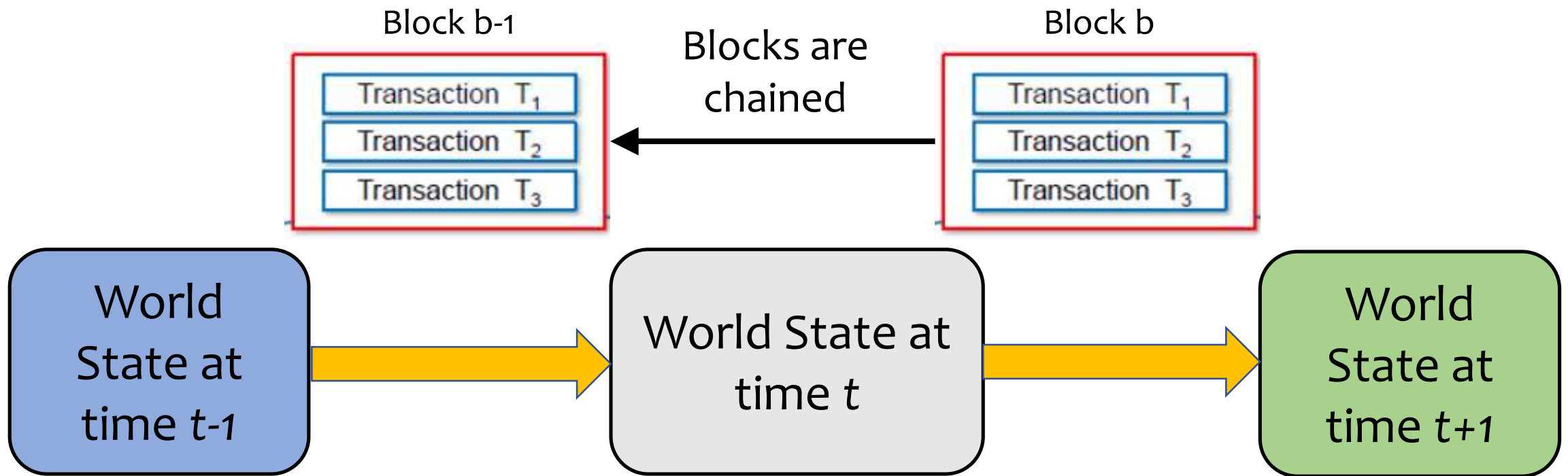
An Ethereum block is a collection of transactions & all the transactions inside a block collectively change the world state



The update rate of world state is equal to the block generation rate

Blockchain

To preserve the transaction history, blocks are strictly ordered and every new block contains a reference to its parent block



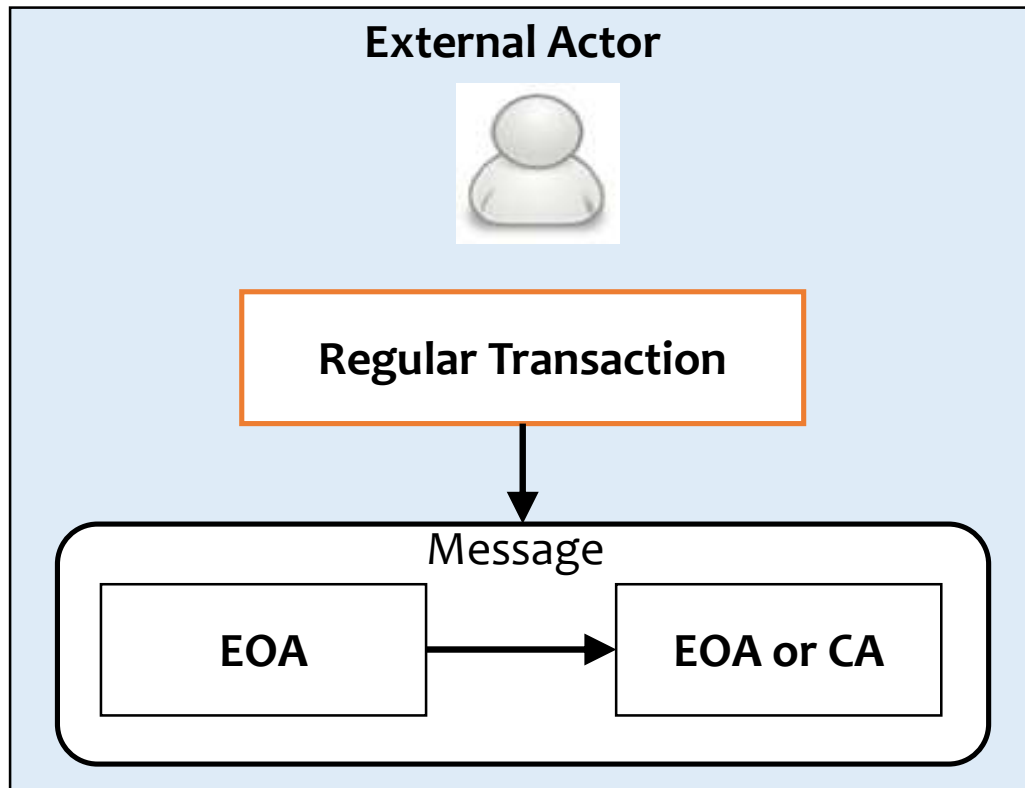
Transactions Change World State

- By default, the Ethereum execution environment is lifeless
 - The state of every account remains the same
- However, any user can trigger action by sending a transaction from an EOA
 - If the destination of the transaction is another EOA, then the transaction may transfer some ether but otherwise does nothing
 - However, if the destination is a contract, then it is activated, and runs its code
- The code of has the ability to:
 - Read and write its own internal storage
 - Access the data of the incoming message (sender, value, data, gas)
 - Send messages (function calls) to other contracts, which can trigger their execution
 - *Send ETH or tokens that it already holds* to other contracts or EOAs
 - Increase the balance of an EOA by sending ETH to it (the EOA's balance in the global state increases)

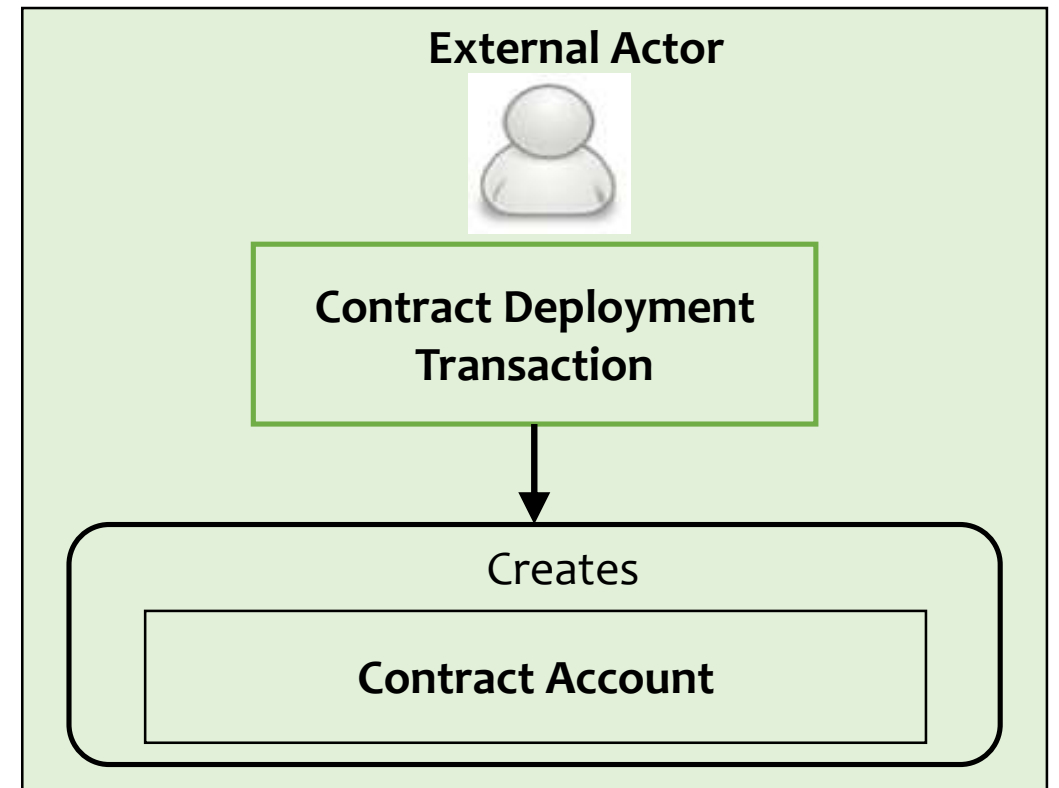
Transactions

Transactions are initiated by EOAs and there are two important categories of Ethereum transactions

Regular Transaction



Contract Deployment Transaction



EIP-1559 Transaction

The receiving address (if an EOA, the regular txn will transfer value. If a CA, the regular txn will execute the contract code). In case of contract deployment txn, this field is empty

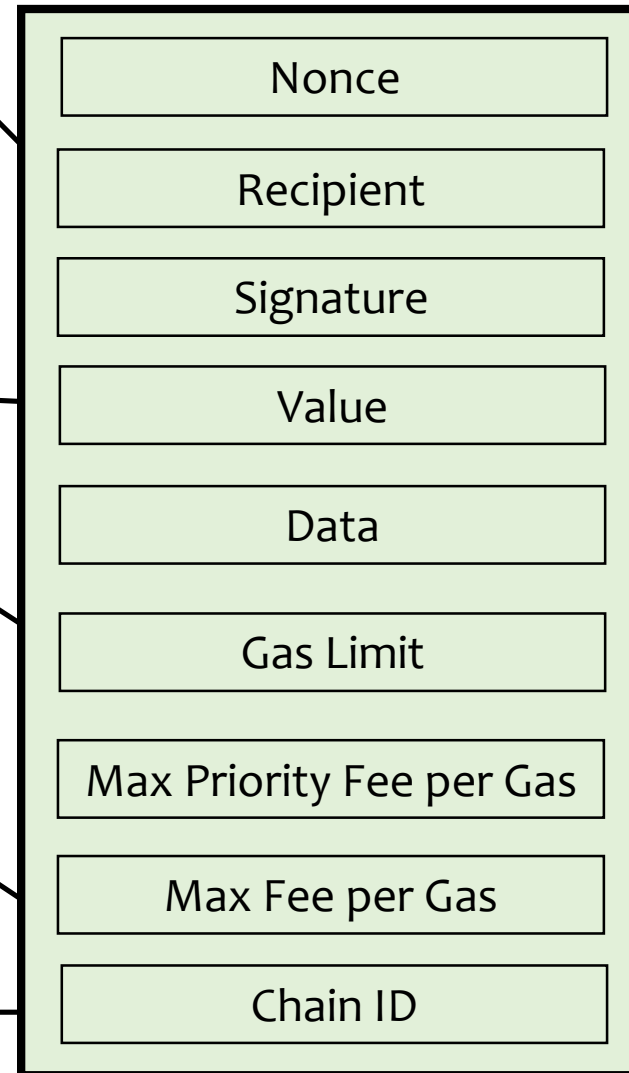
Amount of ETH from sender to recipient (in Wei)

Max amount of gas units to be consumed by txn (generally a guess for complicated txns)

Max price in Gwei per gas unit the sender is willing to pay for the txn

Replay protection on different chains (main chain id is 1)

Transaction



Unique identifier – should be consistent with the account nonce at the time of txn creation (protection against replay attacks)

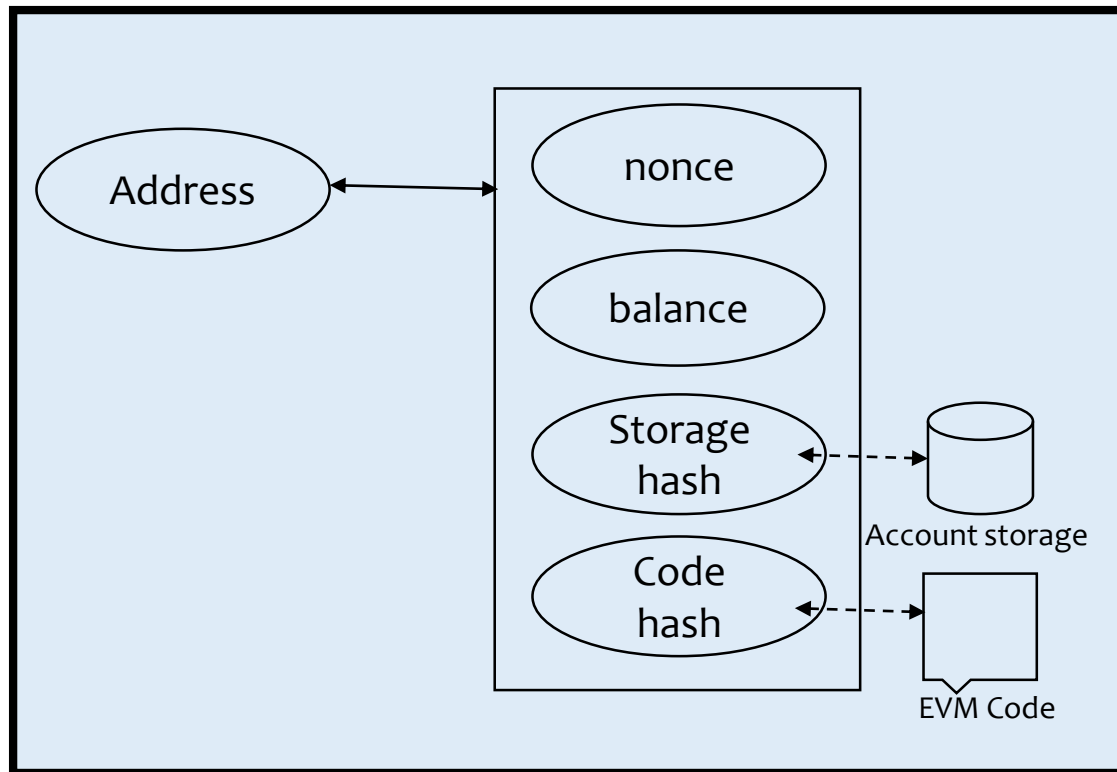
Identifies the sender – txn is signed with sender's private key

Arbitrary data or code (optional). In case of contract deployment txn, this field contains bytecode

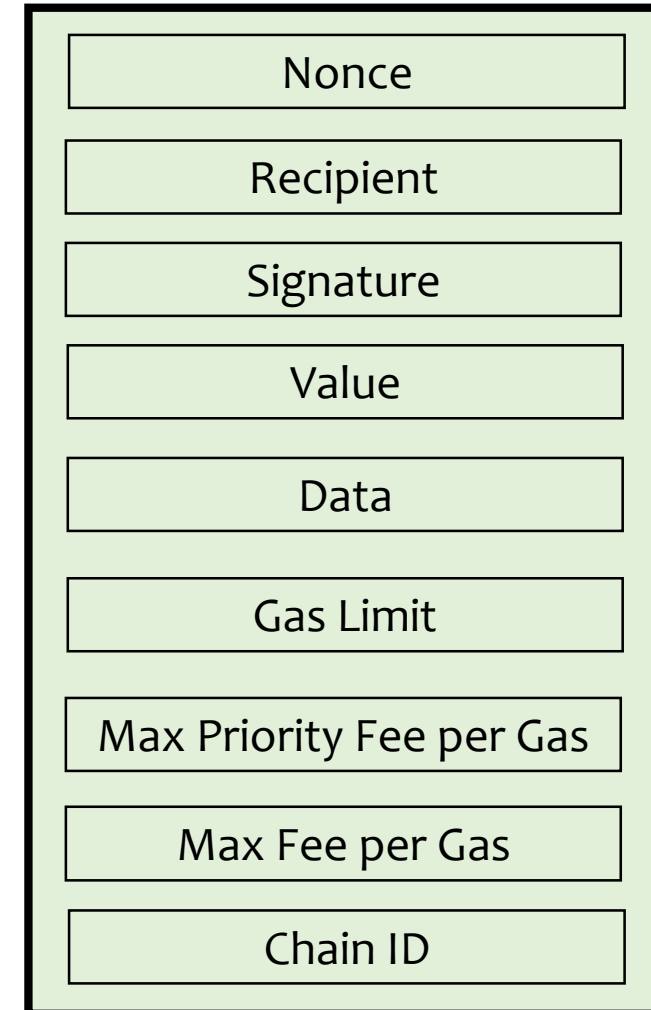
Tip in Gwei per gas unit that the sender is willing to pay to the validator

Accounts & Transactions

Account State



Transaction

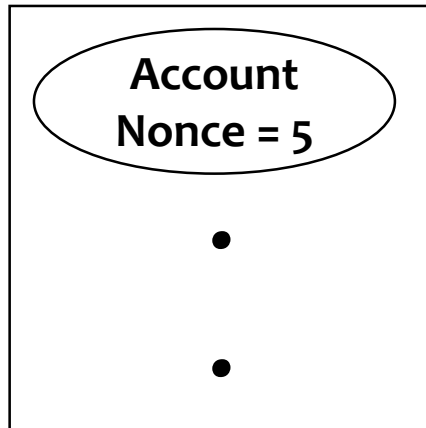


Nonce Role

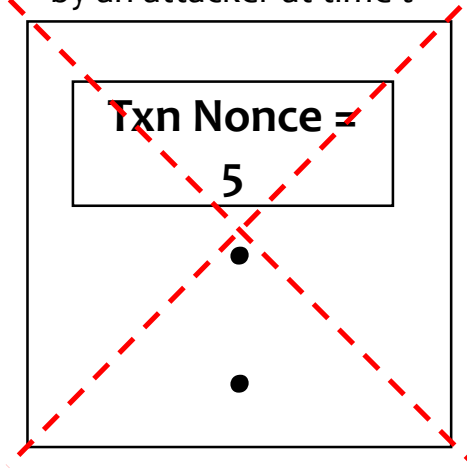
Will this replay attack succeed?

No

Alice's account nonce in world state at time t



Previous txn replayed by an attacker at time t



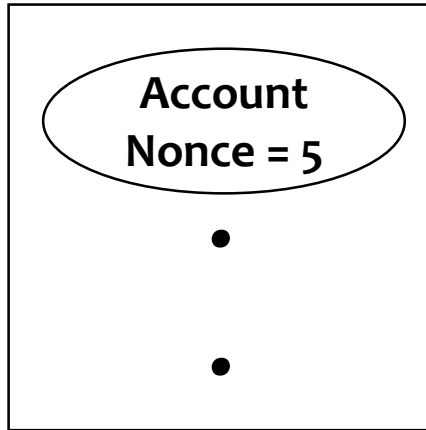
Why?

At time t , account nonce is 5, which means there was a previous txn with the same nonce that was processed by the network and as a result the account nonce was incremented from 4 to 5. The nodes will reject this txn

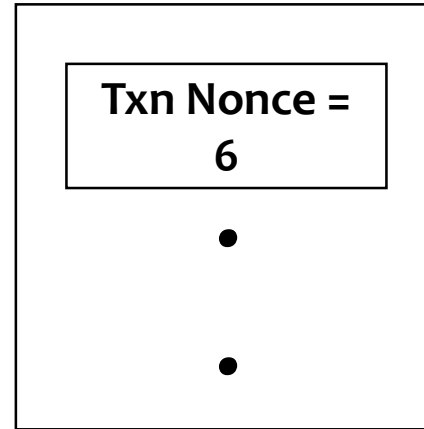
Nonce prevents the replay attacks

Nonce Role

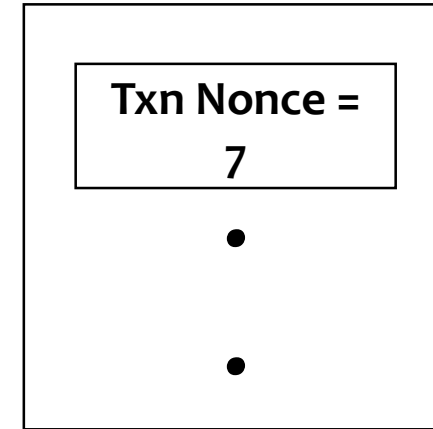
Alice's account nonce in world state at time t



Txn 1 created by Alice at time t



Txn 2 created by Alice at time t



Which txn is valid?

Both txns are valid

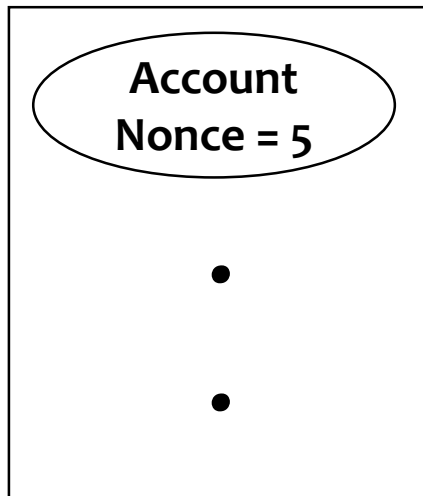
Alice has created two valid txns that are unconfirmed. Nodes can see both the txns. The network will process txn 1 followed by txn 2 and will update account nonce to 7 in the world state. This technique may be used to accelerate txn 1 processing that might get stuck due to lower fee

Nonce allows to choose the order in which transactions will be executed

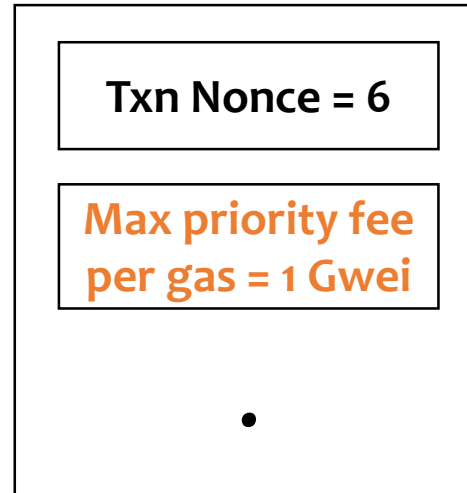
Nonce Role

Two txns are created by Alice at the same time with the same nonce

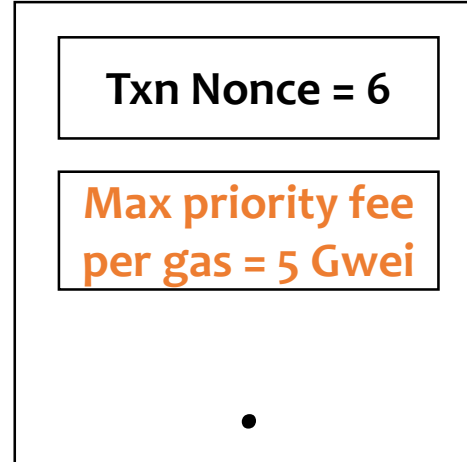
Alice's account nonce in world state at time t



Txn 1 created by Alice at time t



Txn 2 created by Alice at time t



Which txn is valid?

Both txns are valid. However, as soon as one of them gets confirmed, the other txn will become invalid

It is highly likely that the validator will process txn 2 with higher max priority fee (tip). After txn processing, the account state nonce will also be updated to 6 in the world state

This technique may be used to cancel a pending txn that is taking too long to process. Replace it with a new txn with the same nonce but with a higher txn fee

We can cancel an unconfirmed txn by invalidating its nonce

Lecture 23

Tuesday, Apr 21, 2026

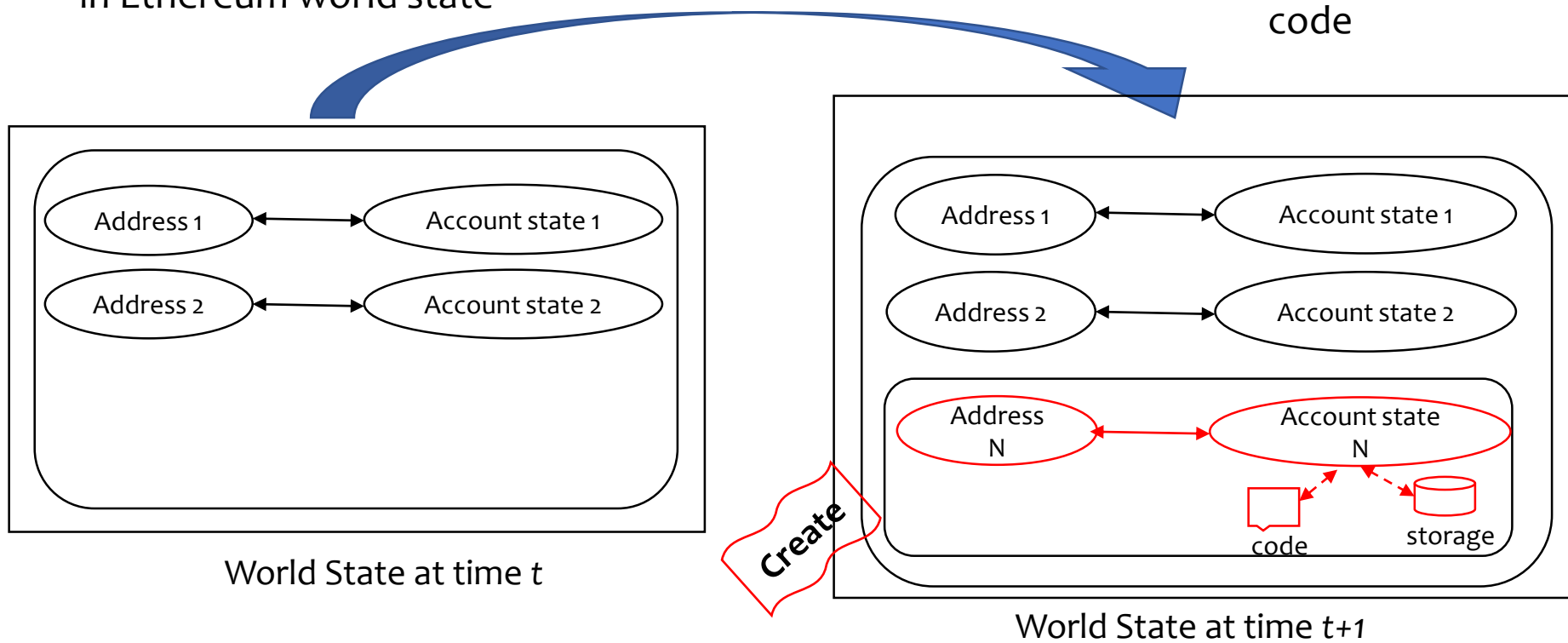
Contract Deployment

Smart contract deployment on blockchain means creation of a new address in Ethereum world state

Contract Deployment Transaction

Code in Data Field

Contract account has a permanent storage, and it is controlled by its EVM code



- Once on blockchain the contract is controlled by its code
- Contracts **cannot** initiate transactions, but can send message calls (function calls) to other contracts or EOAs

Contract Deployment Transaction – Example

0xe5af36d9162a89e2da2b99b5afb5fefe85569bc2b0945ed97a36b7333c2b4bfe

Transaction Hash: 0xe5af36d9162a89e2da2b99b5afb5fefe85569bc2b0945ed97a36b7333c2b4bfe

Status: Success

Block: 11380284 2836829 Block Confirmations

Timestamp: 439 days 21 hrs ago (Dec-03-2020 02:49:24 PM +UTC)

From: 0x4b5057b2c87ec9e7c047fb00c0e406dff2fdacad EOA address

To: [Contract 0xfbddadd80fe7bda00b901fbaf73803f2238ae655 Created] (StrongBlock: Service) The txn from EOA created a new CA

Value: 0 Ether (\$0.00)

Transaction Fee: 0.02988225 Ether (\$93.62)

Gas Price: 0.00000005 Ether (50 Gwei)

Ether Price: \$616.55 / ETH

Gas Limit & Usage by Txn: 597,645 | 597,645 (100%)

Others: Nonce: 1570 Position: 24

Input Data: 0x6080604052604051610cb7380380610cb78339818101604052606081101561002657600080fd5b8101908080519060200190929190805190602001909291908051604051939291908464010000000082111561005a57600080fd5b8382019150602082018581111561007057600080fd5b825186600182028301116401000000008211171561008d57600080fd5b8083526020830192505050908051906020019080838360005b838110156100c15780820151818401526020810190506100a6565b50505050905090810190601f1680156100ee5780820380516001836020036101000a031916815260200191505b506040525050508281600160405180807f656970313936372e70726f78792e696d706c656d656e74617469666e00000000815250601c019050604051809103902060001

Contract code (hexcode) – user writes contract code in high-level language like Solidity, which is converted to bytecode for deployment on blockchain

<https://etherscan.io/tx/0xe5af36d9162a89e2da2b99b5afb5fefe85569bc2b0945ed97a36b7333c2b4bfe>

Size of Contract Code

There is no limit on the size of contract code field

Suppose someone writes a very long code (several hundred MB), includes all the required gas and a huge tip in the txn and then try to deploy this smart contract on Ethereum. Will this attempt succeed?

No

Ethereum avoids this problem by setting an *upper limit on block size in terms of gas units*

Max Block Limit (gas):
30 million gas units (can slightly
exceed this limit)

Target Block Limit (gas):
15 million gas units

A single smart contract that consumes more than 30M gas units cannot be deployed on Ethereum blockchain

Maintaining a Target Gas Limit

- The network adjusts the block base fee to maintain a target block gas limit of 15 million units
- If the last block used **more** than 50% gas, base fee for the next block will **increase**
- If the last block used **less** than 50% gas, base fee for the next block will **decrease**
- The change in the base fee (increase or decrease) from one block to the next is not more than 12.5%
- This base fee is not linked to block generation time (note: Ethereum block is produced every 12 seconds)

Percentage Gas Used and Gas Target

- Percentage gas used in a block = $(\text{Gas used in million} / 30 \text{ M}) \times 100$
- Percentage gas target = $((\text{Gas used in million} - 15 \text{ M}) / 15 \text{ M}) \times 100$
- Percentage gas used in a block can vary between 0% to 100%
- Percentage gas target can vary between -100% to +100%
 - Negative % means gas consumed is less than 15 M
 - Positive % means gas consumed is greater than 15 M

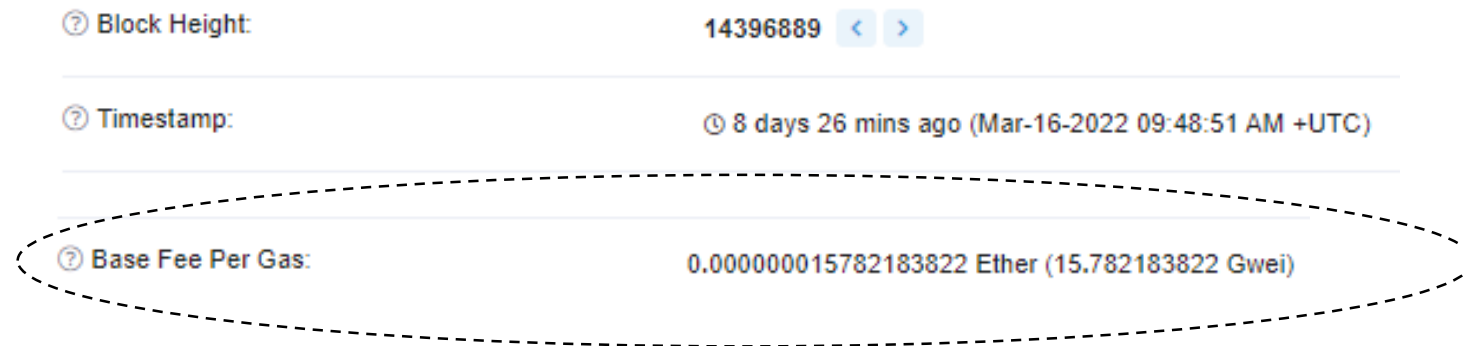
Block Base Fee calculation

Change in base fee for the next block = $(\% \text{ gas used in current block} - 50) / 4$

Base fee for the next block =
Base fee for current block * $[1 + \text{Change in base fee for the next block}]$

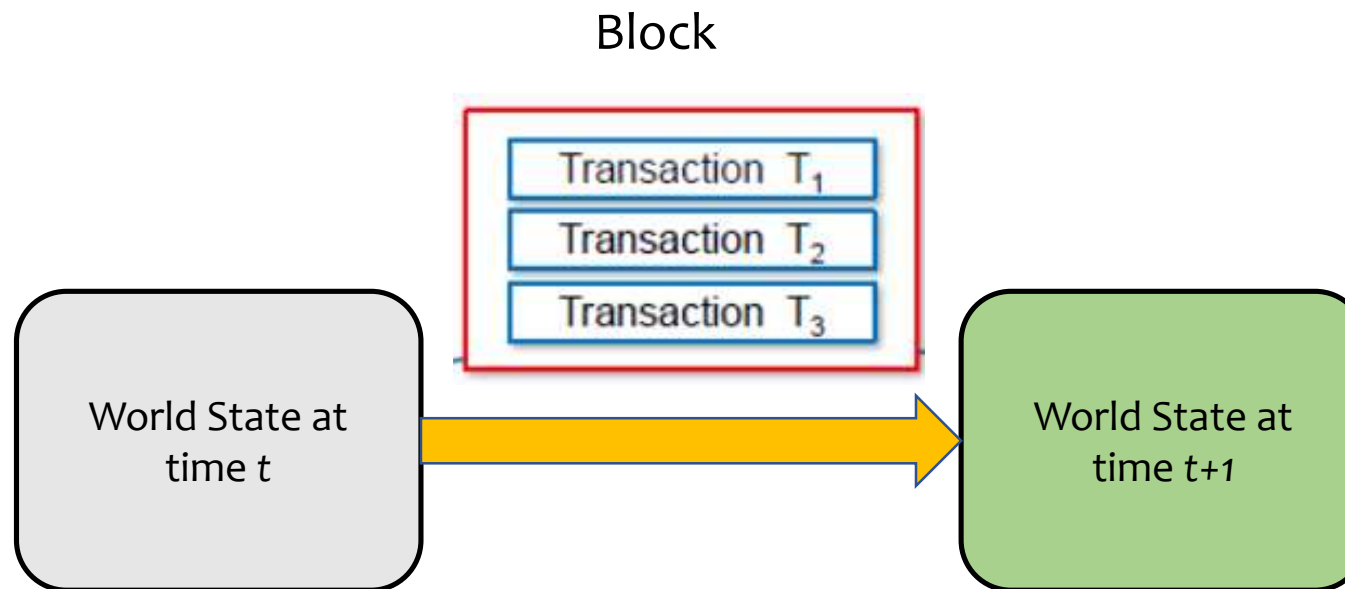
- Base fee for block 14396888 = 17.484043191 Gwei
- Gas used in block 14396888 is 11.06%
- % change in base fee for block 14396889 = $(11.06 - 50) / 4 = -9.375\%$
- Base fee for block 14396889 = $17.484043191 \times [1 + (-9.375 / 100)] = 15.782 \text{ Gwei}$

Block: **14396888**
Gas used = 3.31 M
% gas used = 11.06 %
% gas target = -78 %
Base Fee = 17.484 Gwei



Is Storing Transactions in Blocks Sufficient?

Ethereum block is a collection of transactions and all the transactions inside a block collectively change the world state



Ethereum blocks contain transactions but are transactions sufficient to reproduce and verify everything that happened on blockchain?

No, txns alone are not sufficient

Storing Transactions is Not Sufficient

Suppose Alice wants to send 1 ETH to Bob, she creates a txn and broadcasts it on the network

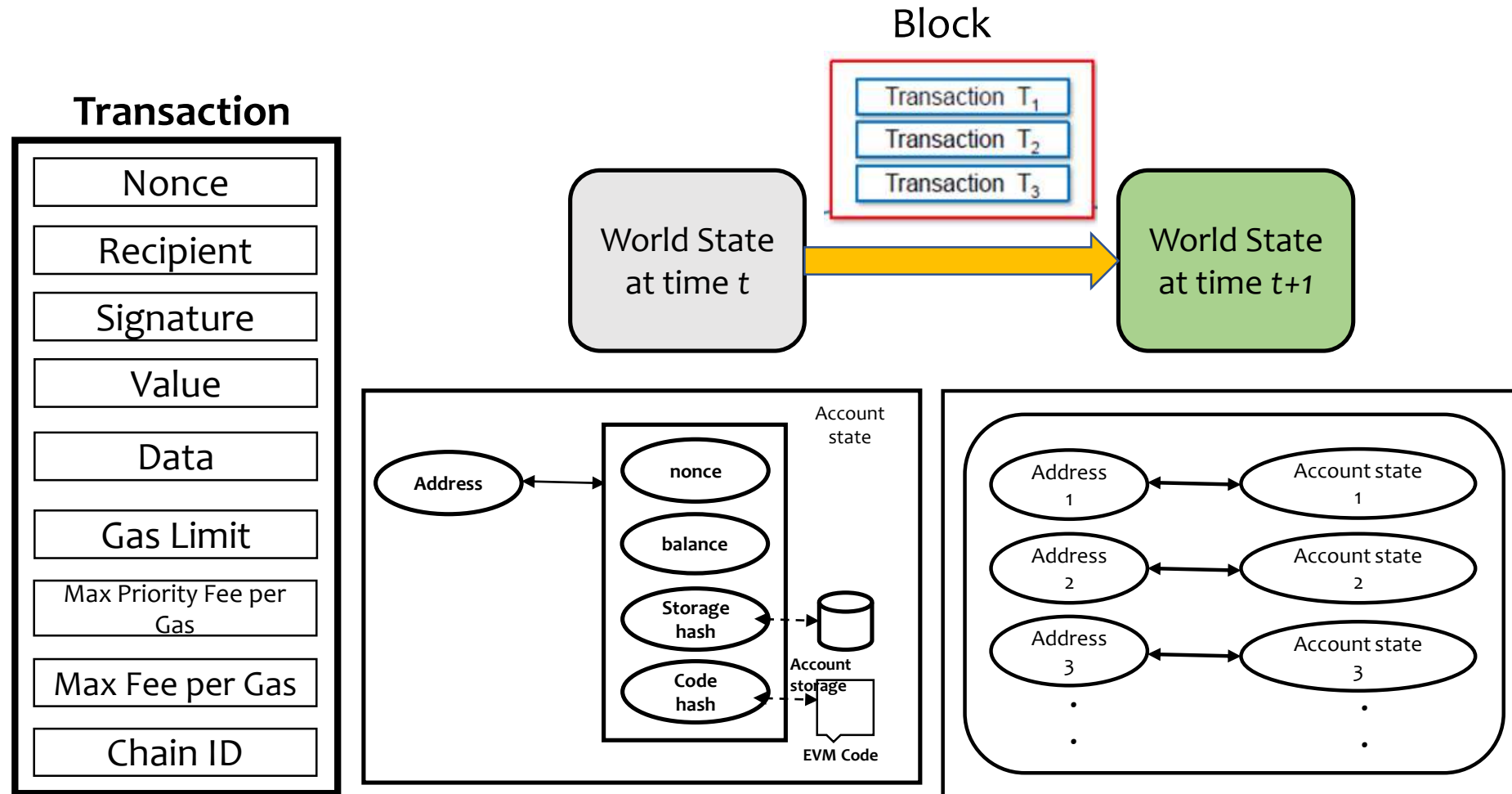
Transaction created by Alice

Nonce=5
Recipient=Bob
Signature=Alice
Value=1ETH
Data (empty)
Gas Limit=21,000
Max Priority Fee per Gas=10Gwei
Max Fee per Gas=100Gwei
Chain ID=1

- Just by looking at this txn, we do not get any information about Alice's account balance
- Suppose Ethereum changes its txn format and adds an account balance field inside the txn. Would this solve the problem?
 - No
- Why?
 - Alice can lie and provide wrong balance
- We do not encounter this problem in Bitcoin because the child txn inputs are always pointing towards the parent UTXOs that Alice wants to spend
- Ethereum's account based model therefore necessitates storing and tracking the world state

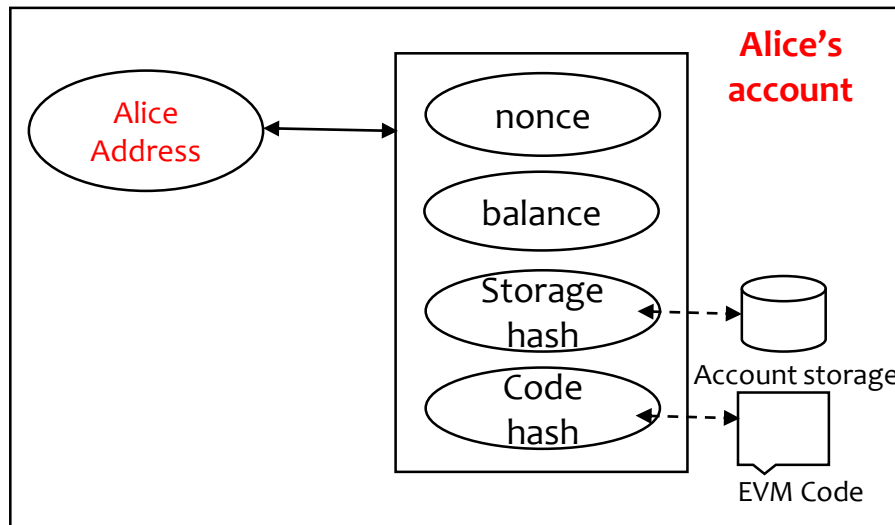
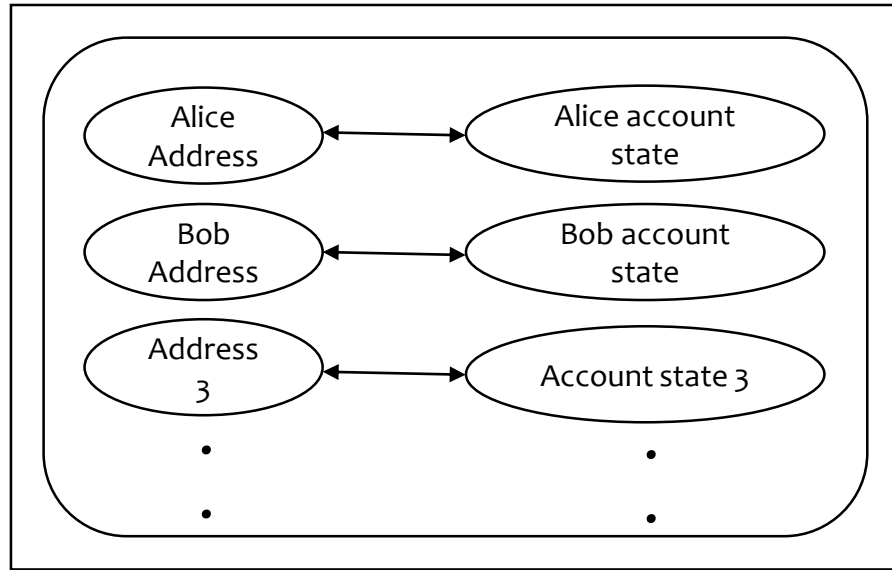
How to keep track of Alice's and all other Ethereum account states?

Where is the World State Stored?

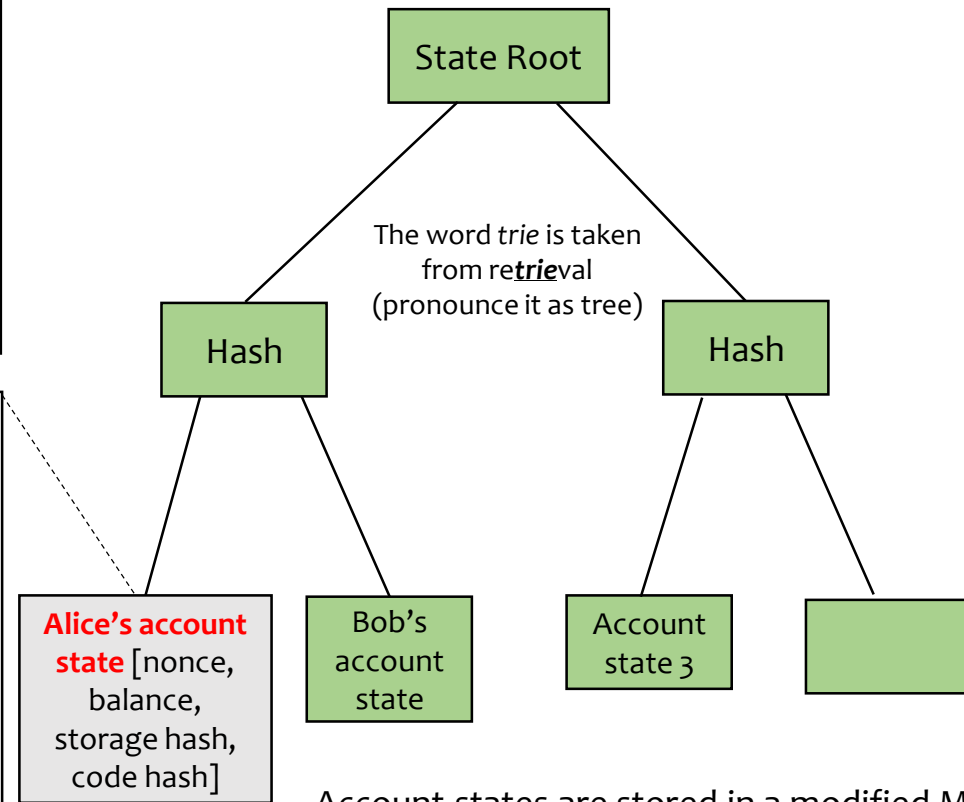


Where is the world state stored?

World State Trie



Solution: Create a world state trie – pack all the Ethereum account states into a deterministic and cryptographically verifiable structure and then store the state root in the block header

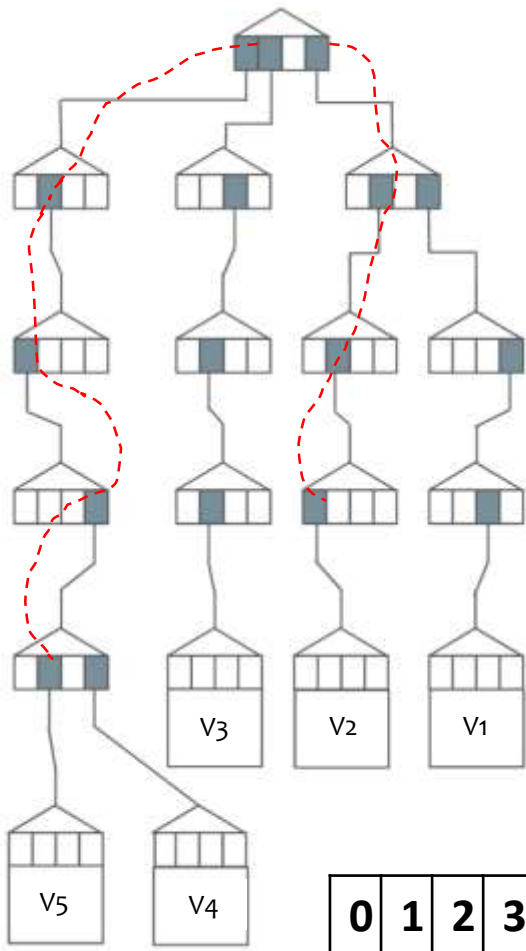


Account states are stored in a modified Merkle Patricia Trie or modified MPT (it is not the same as Merkle tree)

Merkle Patricia Trie (MPT)

Patricia - Practical Algorithm To Retrieve Information Coded In Alphanumeric (1968 paper)

Basic idea: A node's position in a tree defines the **key** with which it is associated



The path to reach value V2:

3-1-1-0

The path to reach value V5:

0-1-0-3-1

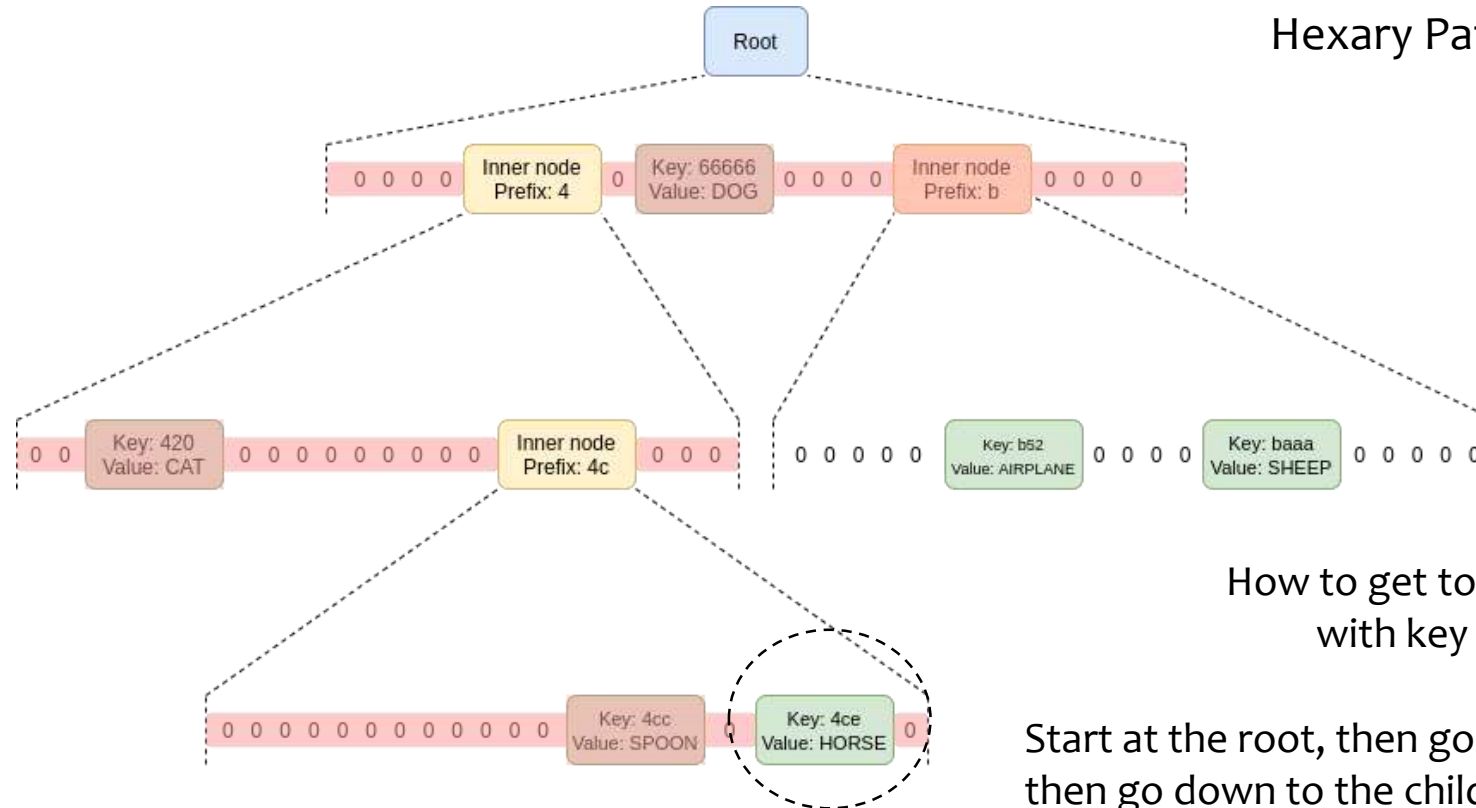
Ethereum uses hexary Patricia tree with 16 choices at each node (a modified version of MPT). It also uses hashing so that the root node becomes a cryptographic fingerprint of the entire data structure. The trie is implemented as a key-value database

<https://blog.ethereum.org/2019/12/30/eth1x-files-state-of-stateless-ethereum/>

Lecture 24

Thursday, Apr 23, 2026

Hexary MPT



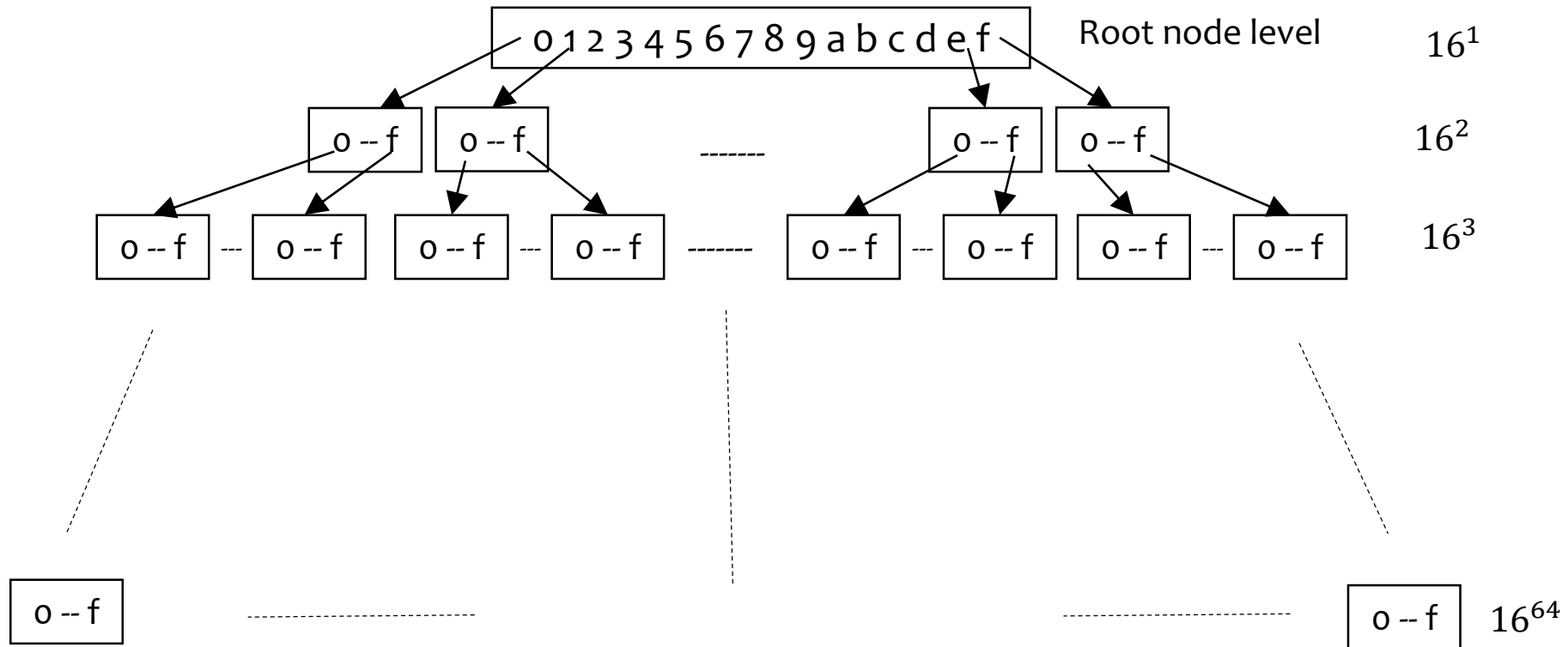
Hexary Patricia tree with 16 choices at each node

How to get to the node with key **4ce**?

Start at the root, then go down to the child at position **4**, then go down to the child at position **c** (remember: **c** = 12 in hexadecimal), and then go down again to the child at position **e**

In Ethereum, key to reach an account state in the MPT is the 256-bit hash of its Ethereum address

Hexary MPT



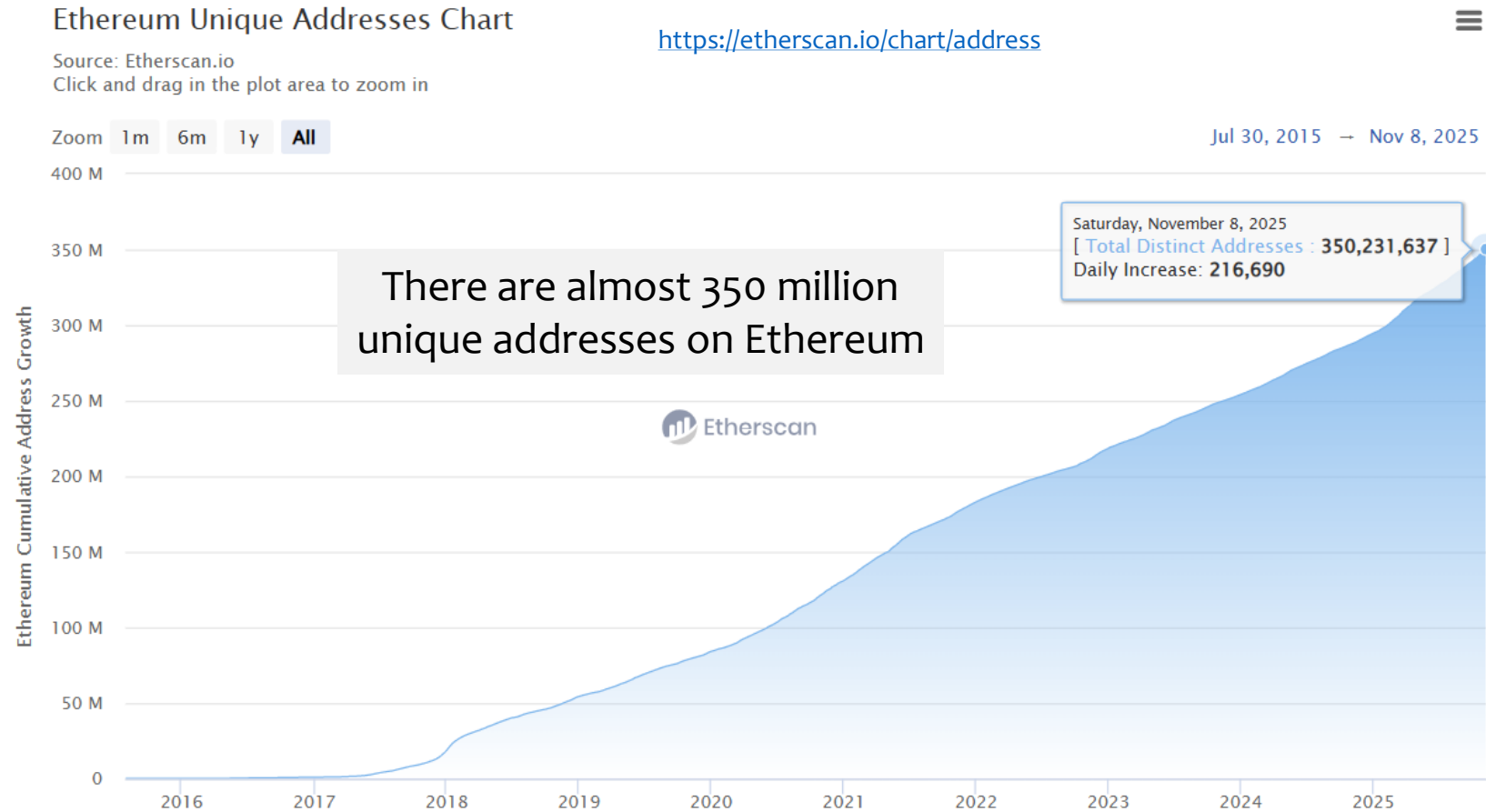
Total positions or keys at the 64^{th} level are $16^{64} = 2^{256}$

In a tree like structure, a leaf has
no child

Do we need all the 2^{256} leafs (keys)?

No, because total Ethereum addresses
are far less than 2^{256}

Ethereum Unique Addresses



- 350 million is approximately equal to $2^{28.39}$
- This allows Ethereum to significantly compress the trie with a lot less leaf nodes because there are several branches without child nodes in the hexary tree

Ethereum's Modified MPT

- Ethereum defines three types of nodes in its modified MPT
 - **Leaf Node**: A node with no child
 - **Extension node**: A node with only one child node
 - **Branch node**: A node with more than one child node (any number from 2 to 16)
- This distinction of nodes allow Ethereum to compress the MPT
 - Leaf node stores a key and a value (value is account state)
 - Extension node also stores a key and a value (value is the hash of the child node)
 - Branch node has 17 items that include 16 slots (for hex digits 0-f) and a value (either empty or non-empty if some key ends here). The next nibble of the key determines which branch slot to follow

Ethereum's Modified MPT

- To differentiate between the leaf and the extension nodes (because both store two items), following prefixes are used:
 - 0000 (0x0): Extension node with even number of nibbles
 - 0001 (0x1): Extension node with odd number of nibbles
 - 0010 (0x2): Leaf node with even number of nibbles
 - 0011 (0x3): Leaf node with odd number of nibbles
- Computers cannot store half of byte, so padding is also used with the prefix to convert the key into bytes

Even nibbles

Example: [4,5,6,7]

- Extension node: After prefix padding
[00,45,67]
- Leaf node: After prefix padding
[20,45,67]

Odd nibbles

Example: [5,6,7,8,9]

- Extension node: After prefix padding
[15,67,89]
- Leaf node: After prefix padding
[35,67,89]

Simplified Modified MPT Example

Develop an Ethereum-like Modified MPT for the following simplified world state

There are four accounts (4 addresses) so there will be a total of 4 leaf nodes. All the addresses are 7 nibble long

Simplified World State							
Account Keys							Values
a	7	1	1	3	5	5	45.0 ETH
a	7	7	d	3	3	7	1.00 WEI
a	7	f	9	3	6	5	1.1 ETH
a	7	7	d	3	9	7	0.12 ETH

Root: Extension Node		
Prefix	Shared nibble(s)	Next node
0x00	a7	

Only one child that is a branch node

- We develop MPT from the root node and start from the first nibbles of the account addresses
- All the accounts have the same first two nibbles i.e., **a7**. Therefore, we group these two nibbles (compression)
- The third nibbles of the addresses are **1, 7, f, 7** respectively. The next node would therefore be a branch node and since there is only one child of root node, the root node is an extension node
- Root node key contains even number of nibbles; therefore, we also add a prefix **0x00**
- The complete key of the root node is **0x00a7**

Simplified Modified MPT Example

Simplified World State							
Account Keys							Values
a	7	1	1	3	5	5	45.0 ETH
a	7	7	d	3	3	7	1.00 WEI
a	7	f	9	3	6	5	1.1 ETH
a	7	7	d	3	9	7	0.12 ETH

The branch node originating from the root node has 17 items, but it has only three child nodes that originate from the positions **1**, **7**, and **f**

Branch Node																
0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f	Value
	●						●								●	

The child node originating from position **1** of the branch node can be declared a leaf node for account 1 by grouping the remaining four nibbles **1355** (compression). We add a prefix **0x20** since the leaf node has even number of nibbles. The key for this leaf node becomes **0x201355** and the value is **45.0 ETH**

Leaf Node		
Prefix	Key-end	Value
0x20	1355	45.0 ETH

Similarly, the child node originating from position **f** of the branch node can also be declared a leaf node for account 3 by grouping the remaining four nibbles **9365** (compression). We add a prefix **0x20** since the leaf node has even number of nibbles. The key for this leaf node becomes **0x209365** and the value is **1.1 ETH**

Leaf Node		
Prefix	Key-end	Value
0x20	9365	1.1 ETH

The child node originating from position **7** has common nibbles, therefore it is an extension node. We group two common nibbles **d3** (compression) and add a prefix **0x00** since the extension node has even number of nibbles. The key for this extension node becomes **0x00d3**

Extension Node		
Prefix	Shared nibble(s)	Next node
0x00	d3	●

Simplified Modified MPT Example

Simplified World State							
Account Keys							Values
a	7	1	1	3	5	5	45.0 ETH
a	7	7	d	3	3	7	1.00 WEI
a	7	f	9	3	6	5	1.1 ETH
a	7	7	d	3	9	7	0.12 ETH

Extension Node		
Prefix	Shared nibble(s)	Next node
0x00	d3	

The child node of this extension node (0x00d3) is a branch node because the second last nibbles of accounts 2 and 4 are different: 3 and 9 respectively

Branch Node																
0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f	Value

There are two child nodes of this branch node and both of them are leaf nodes

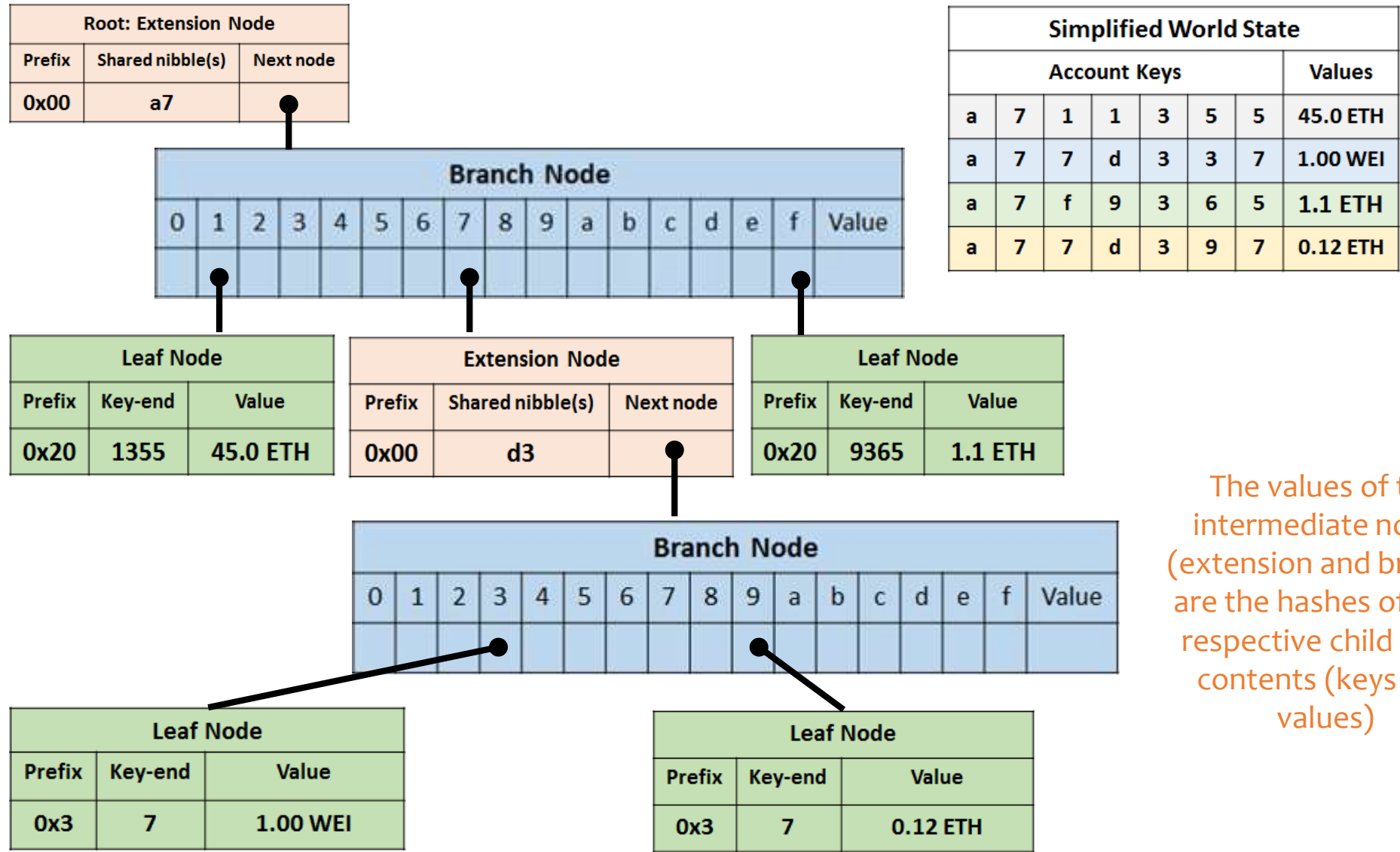
The child node originating from position 3 is a leaf node for account 2. We add a prefix 0x3 since the leaf node has odd number of nibbles (only one). The key for this node becomes 0x37 and the value is 1.00 WEI

Leaf Node		
Prefix	Key-end	Value
0x3	7	1.00 WEI

The child node originating from position 9 is a leaf node for account 4. We add a prefix 0x3 since the leaf node has odd number of nibbles (again only one). The key for this node becomes 0x37 and the value is 0.12 ETH

Leaf Node		
Prefix	Key-end	Value
0x3	7	0.12 ETH

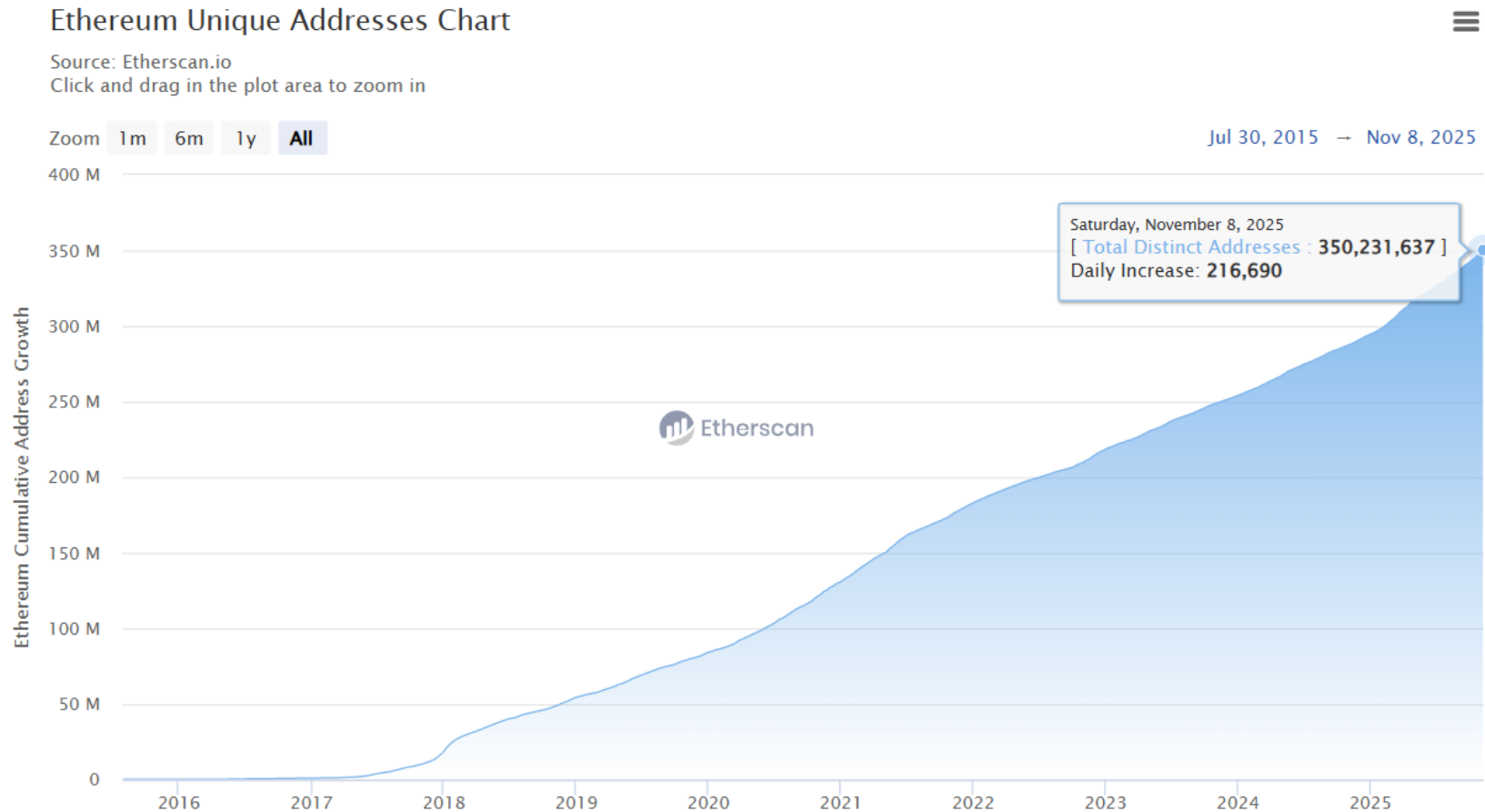
Simplified Modified MPT Example



The values of the intermediate nodes (extension and branch) are the hashes of their respective child node contents (keys and values)

Ethereum Unique Addresses

Is the requirement to keep track of all the account states makes Ethereum less efficient?

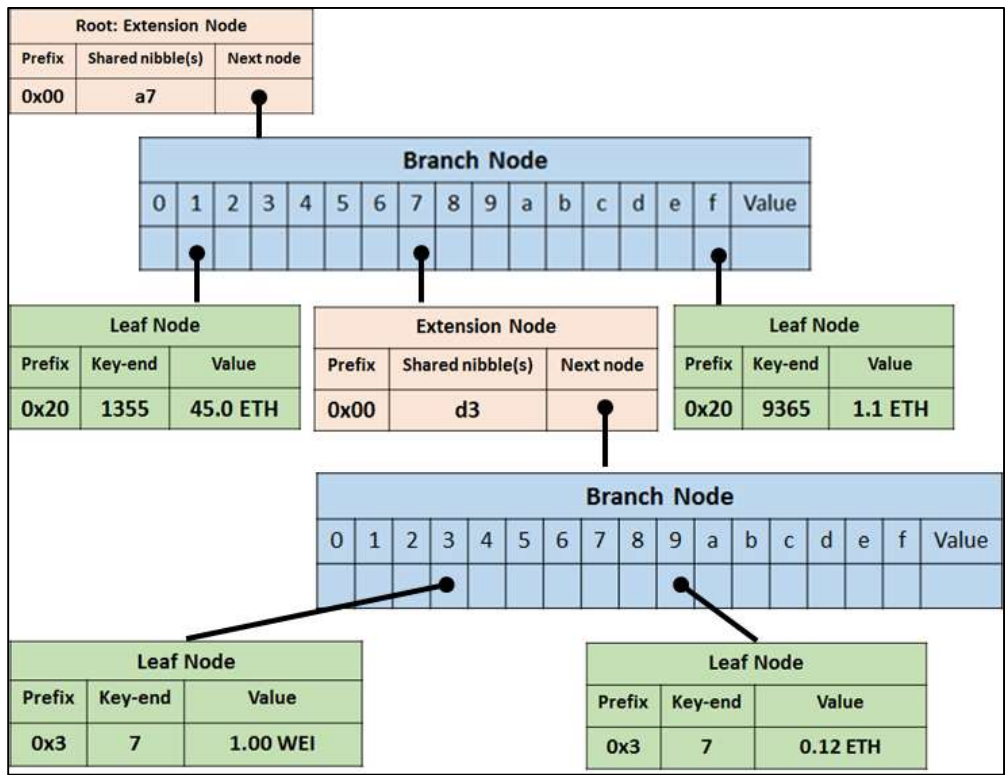


Daily Active Ethereum Addresses



- Daily active Ethereum addresses are only around 0.5 M to 0.8 M. This is just a small fraction of total Ethereum addresses $(0.8/225) \times 100 = 0.355\%$
- The number of accounts touched in a single block is even small (few hundred)
- After every block only a small part of the tree needs to be changed. Thus, between two adjacent blocks the vast majority of the trie remains the same, which makes the problem manageable

State Trie Update – Example



Simplified World State							
Account Keys							Values
a	7	1	1	3	5	5	45.0 ETH
a	7	7	d	3	3	7	1.00 WEI
a	7	f	9	3	6	5	1.1 ETH
a	7	7	d	3	9	7	0.12 ETH

Suppose a new (5th) Ethereum account (0xa351234) is created in our simplified world state, and it receives 10 ETH from 0xa711355 account

Simplified World State							
Account Keys							Values
a	3	5	1	2	3	4	10.0 ETH
a	7	1	1	3	5	5	35.0 ETH
a	7	7	d	3	3	7	1.00 WEI
a	7	f	9	3	6	5	1.1 ETH
a	7	7	d	3	9	7	0.12 ETH

Create a new state trie?

State Trie Update – Example

Simplified World State							
Account Keys							Values
a	3	5	1	2	3	4	10.0 ETH
a	7	1	1	3	5	5	35.0 ETH
a	7	7	d	3	3	7	1.00 WEI
a	7	f	9	3	6	5	1.1 ETH
a	7	7	d	3	9	7	0.12 ETH

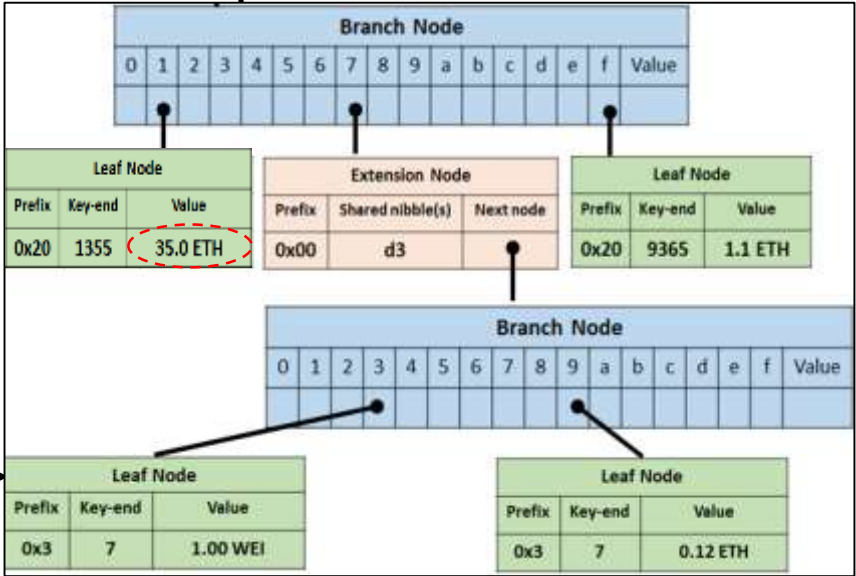
Root: Extension Node		
Prefix	Shared nibble(s)	Next node
0x1	a	

Root node is once again an extension node with only one child – child is a branch node

Branch Node																
0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f	Value

Leaf Node		
Prefix	Key-end	Value
0x3	51234	10.0 ETH

This portion of the trie remains the same (account 0xa711355 state is updated)



State Trie Update – Example

Suppose another (6th) Ethereum account (0x2f5d9eb) is created in our simplified world state, and it receives 7.0 ETH from 0xa351234 account

Root node is a branch node with two child nodes originating from positions 2 and a

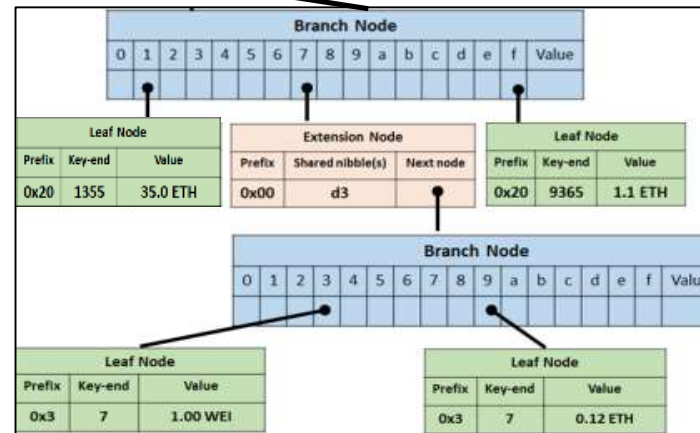
Branch Node															
0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f

Leaf Node		
Prefix	Key-end	Value
0x20	f5d9eb	7.0 ETH

Branch Node															
0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f

Leaf Node		
Prefix	Key-end	Value
0x3	51234	3.0 ETH

This portion of the trie remains the same (account 0xa351234 state is updated)



Simplified World State							
Account Keys							Values
2	f	5	d	9	e	b	7.0 ETH
a	3	5	1	2	3	4	3.0 ETH
a	7	1	1	3	5	5	35.0 ETH
a	7	7	d	3	3	7	1.00 WEI
a	7	f	9	3	6	5	1.1 ETH
a	7	7	d	3	9	7	0.12 ETH

Only a small part of the trie is updated, which makes the problem manageable

Why hash of addresses as path key?

- Ethereum account addresses are 160-bit (40 nibble) long which means $16^{40} = 2^{160}$ distinct leaves can be constructed. Current unique Ethereum addresses are roughly 2^{28} , which is a lot smaller number than 2^{160}
- Why Ethereum uses 256-bit (64 nibble) hash of account addresses to build a state trie?
- When account address is hashed, it randomizes its string representation and contributes to more random distribution of branching in the state trie
- This makes it impossible to construct account addresses in a way that degrades the resulting trie by containing long paths with minimal branching

Ethereum Tries

In Ethereum, the universal data structure is Merkle Patricia Trie (MPT)

State Trie

It is a representation of the world state containing all the addresses of all the accounts and maps them to the account states (MPT structure)

Transaction Trie

With every new block, a new transaction trie (MPT) is built up, containing all the transactions from the block

Transaction Receipt Trie

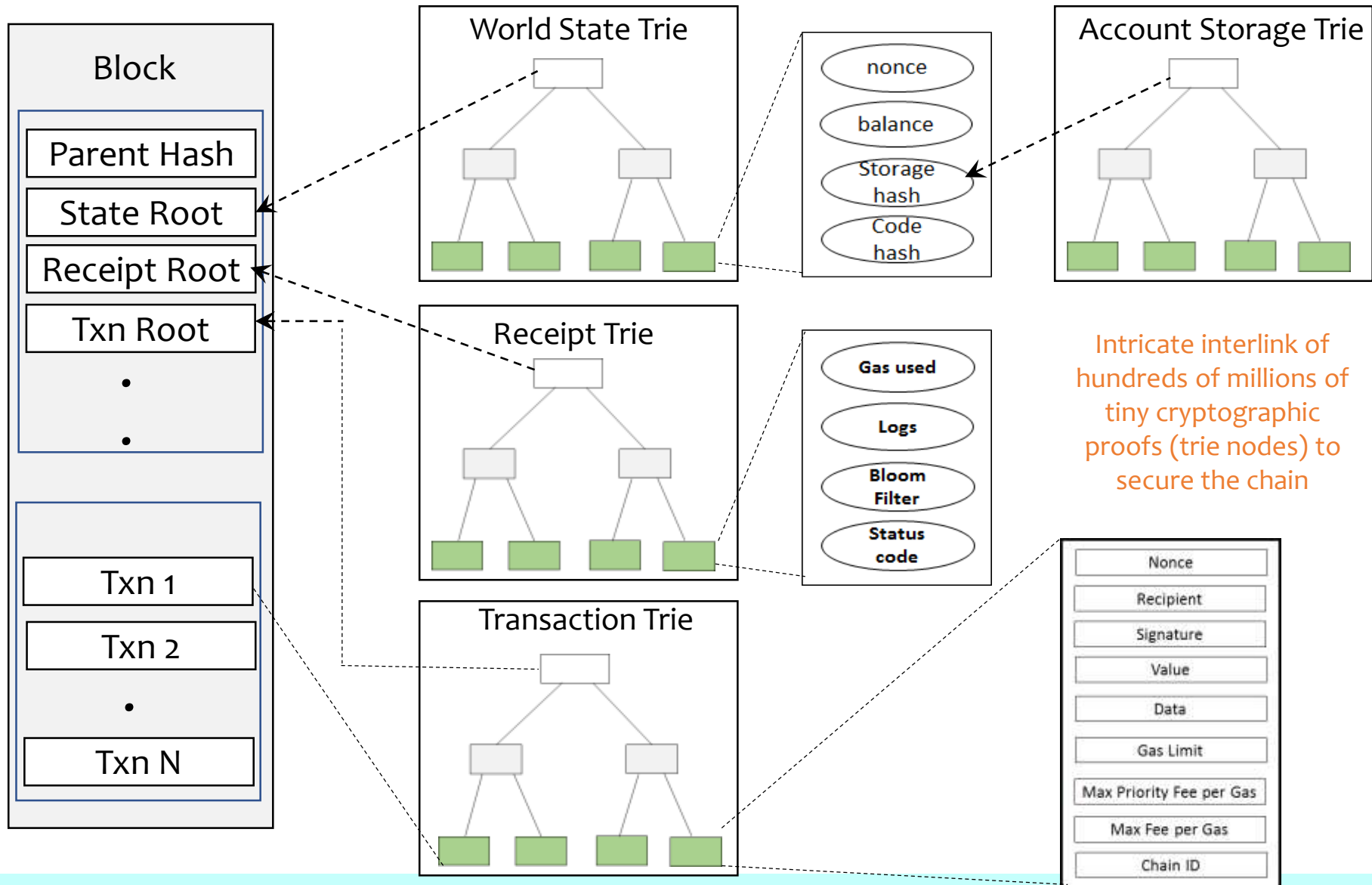
Transaction receipts are used to record the outcomes of the transaction execution. Information such as transaction status code, cumulative gas used in the block, logs (logger's address, log data, log topic), and a bloom filter (256-bit hash of logs) are used to build this trie (MPT structure)

Storage Trie

For every account, there is a storage trie, which contains the storage contents and storage addressing scheme of this account. Every account state in the world trie contains the root of the storage trie of this account (storagehash)

Roots of state trie, transaction trie, and transaction receipt trie are hashed (Keccak-256 hash) and included in the block header

State Tries in Block Header



Ethereum Full Node

Running an Ethereum full node requires

- Downloading all the blocks
- Downloading the world state trie (a data structure with 350 M accounts and billions of MPT nodes)
- Cross-checking the information in blocks and **executing the codes** to validate states



Top 10 Countries [View All Nodes](#)

Total 11,219 nodes found

#	Countries	Last 24 Hours	Last 24 Hours	Last 7 Days
1	United States	5,073 (44.87%)	▼ 82.76%	▼ 49.85%
2	China	1,947 (17.22%)	0.00%	▼ 52.16%
3	Germany	756 (6.69%)	▼ 65.00%	▼ 47.32%
4	Japan	438 (3.87%)	▼ 90.91%	▼ 33.55%
5	France	410 (3.63%)	▼ 52.94%	▼ 48.42%
6	Canada	362 (3.20%)	▼ 42.86%	▼ 52.26%
7	Russia	294 (2.60%)	▼ 98.30%	▼ 36.80%
8	Australia	261 (2.31%)	▼ 98.47%	▲ 43.52%
9	United Kingdom	247 (2.18%)	▼ 33.33%	▼ 50.68%
10	Singapore	148 (1.31%)	▼ 98.65%	▼ 4.82%

- There are roughly 11,000 synchronized Ethereum full nodes spread across the world, which are validating the transactions and constantly updating the world state trie
- The interest in a blockchain project (health) may be gauged from the number of full nodes in the network

Full Node Limitation

Suppose Alice's account balance on Mar 14 was 5 ETH, and then she made the following transactions:

- Sent 0.1 ETH to Bob on Mar 15
- Sent 0.2 ETH to Kevin on Mar 16
- Received 0.6 ETH from her mom on Mar 17
- Paid 2 ETH to buy NFT on Mar 18

Alice's account balance changed in the following way

Mar 14	Mar 15	Mar 16	Mar 17	Mar 18
5 ETH	4.9 ETH	4.7 ETH	5.3 ETH	3.3 ETH

Assume, Alice has not made any other transaction after Mar 18

Suppose you are running a full node and want to check Alice's account balance. What value will you find in the state trie?

3.3 ETH

Suppose you want to know Alice's account balance on Mar 15. Will you be able to find it in the state trie?

No

Why?

State trie only stores the updated information about the accounts, there is no archive of all historical states

Ethereum Node Types

Full Node

- Stores full blockchain data
- Participates in block validation, verifies all blocks and states
- Serves the network and provides data on request
- All states can be derived from a full node

Light Node

- Stores the headers and requests everything else
- Can verify the validity of the data against the state roots in the block headers
- Useful for low-capacity devices (embedded devices or mobile phones)

Archive Node

- An archive node stores everything kept in the full node
- Builds and stores an archive of all the historical states
- This node contains the complete information including all the past and present states of all the Ethereum accounts

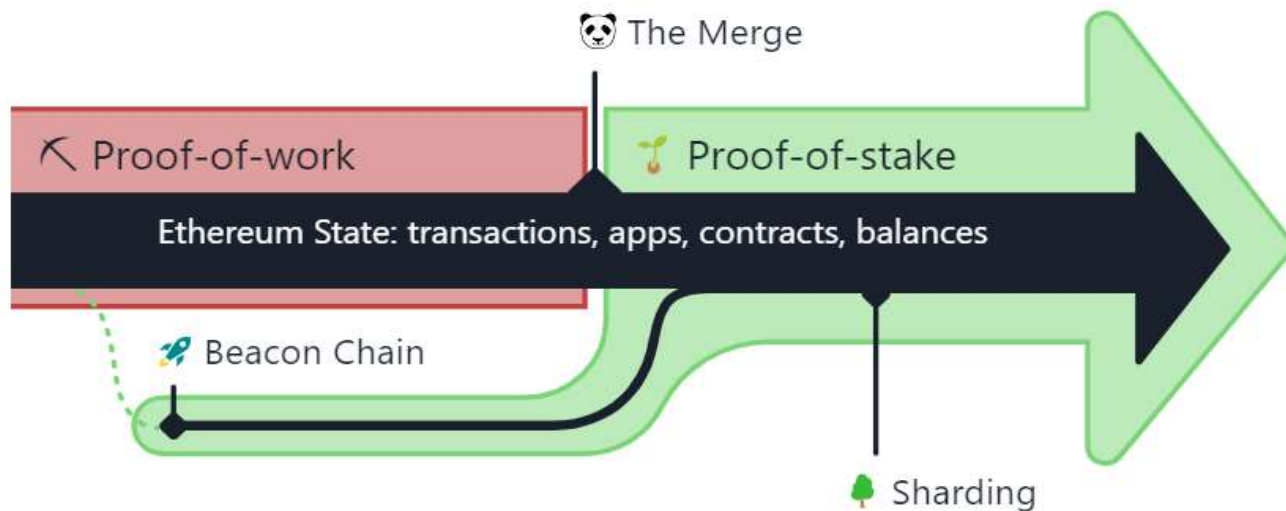
Bitcoin has full nodes and light nodes but there is no concept of archive node in Bitcoin

The Merge – Switch to PoS Consensus

Ethereum switched from PoW to PoS on Sep 15, 2022, and this event is known as ‘The Merge’

Ethereum Mainnet was launched on July 30, 2015, with PoW consensus layer. On Dec 1, 2020, the Beacon Chain (with new PoS consensus layer) was created as a separate blockchain to Mainnet, running in parallel

The Merge was the joining of the original execution layer of Ethereum with the PoS Beacon Chain



The Merge represents the formal adoption of the Beacon Chain as the new consensus layer to the original Mainnet execution layer

PoW vs PoS

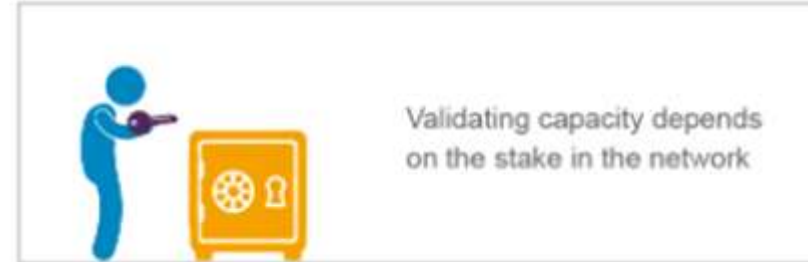
Proof of Work

VS

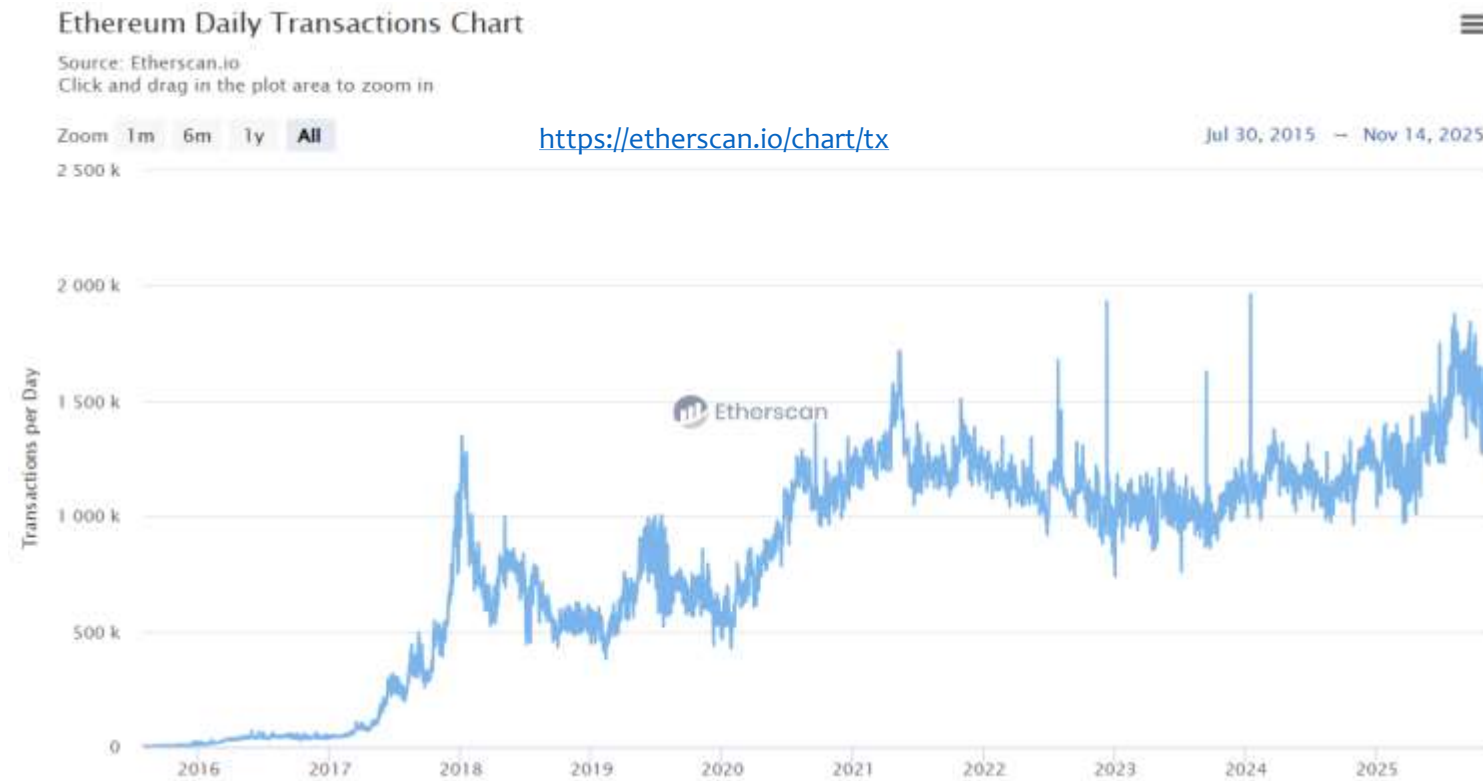
Proof of Stake

Miners

Validators



Scalability



- Ethereum Throughput = $1.5\text{M}/86,400 = 17.36 \text{ txns/s (tps)}$
- Ethereum throughput is 3 to 4 times greater than Bitcoin's throughput

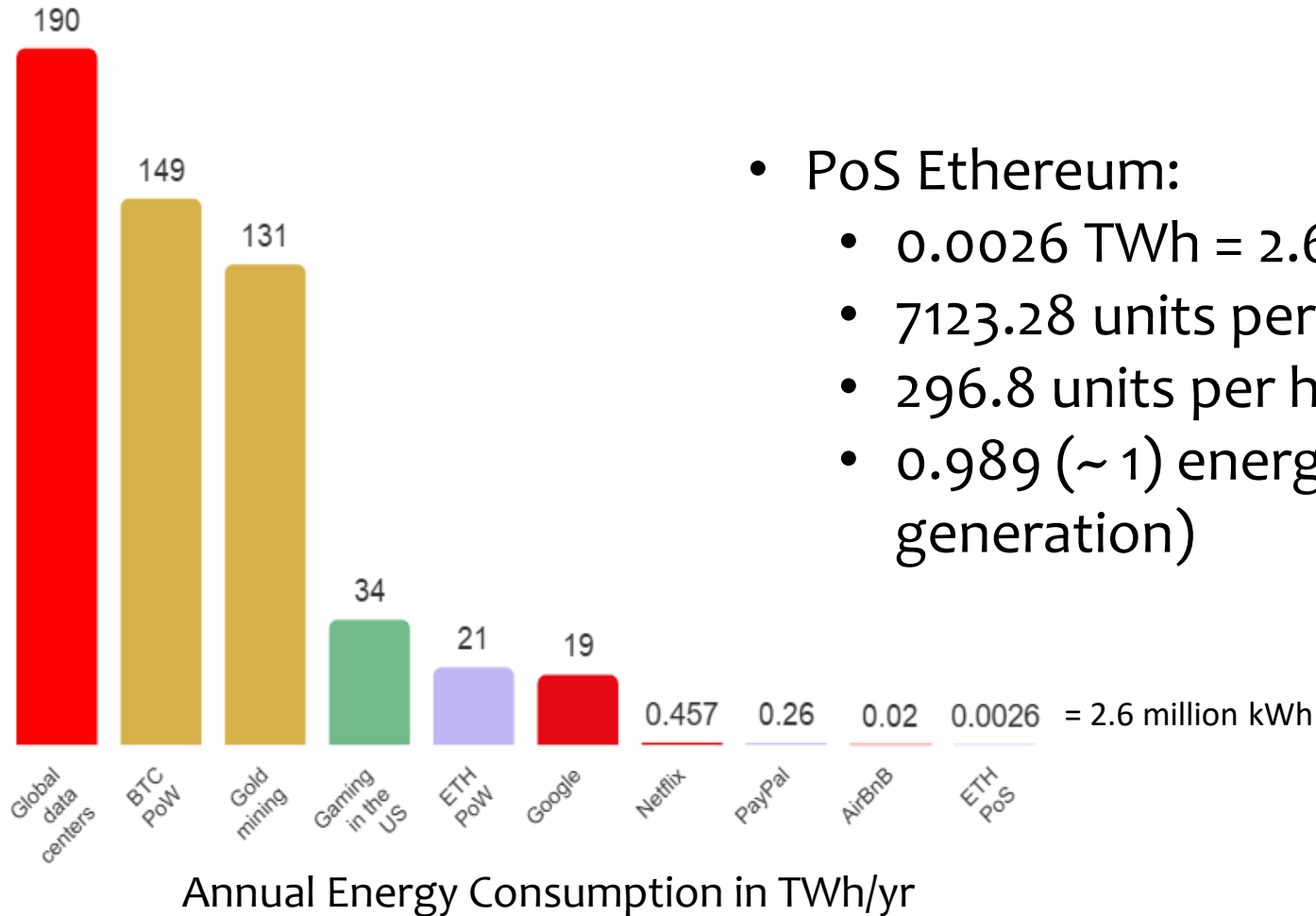
Ethereum also suffers from throughput and scalability problems, and these issues are much more serious because Ethereum is an enabling platform for stablecoins, Web 3 and decentralized applications

- The switch from PoW to PoS did not directly solve the scalability issue
- The throughput is almost the same because block size and block generation rate were kept same
 - Block size: around 100 to 200 kB
 - Block generation rate: 12 s

Reasons for the Merge

- Ethereum's switch from PoW to PoS did not change its block size or block generation rate
- So, why did Ethereum made this switch?
- **Massive energy reduction**
 - PoS reduced Ethereum's energy usage by >99.9%, eliminating the environmental cost of PoW
- **Stronger economic security**
 - Attacks become more expensive because malicious validators ETH can be slashed
- **Better long-term sustainability**
 - PoS requires far lower creation of ETH, allowing ETH to become deflationary
- **Broader decentralization**
 - No dependence on any specialized hardware enabling many more participants to join validation
- **Foundation for future scalability**
 - PoS is required for sharding

Ethereum PoS Energy Consumption



- PoS Ethereum:
 - 0.0026 TWh = 2.6 million energy units per year
 - 7123.28 units per day
 - 296.8 units per hour
 - 0.989 (~ 1) energy units per block (12s block generation)

<https://ethereum.org/en/energy-consumption/#:~:text=This%20yielded%20an%20estimate%20of,regional%2Dspecific%20carbon%20intensity%20factors.>

Ethereum PoS

ETH are locked in a smart contract called Beacon Deposit Contract deployed on Ethereum blockchain

[Beacon Deposit Contract](#) [Source Code](#) [# Smart Contract](#)

Overview

ETH BALANCE
72,664,593.10715194469537452 ETH

ETH VALUE
\$248,947,520,852.31 (@ \$3,425.98/ETH)



<https://etherscan.io/address/0x00000000219ab540356cbb839cbe05303d7705fa>

Ethereum PoS – Validators

Validators are nodes who have staked 32 ETH to participate in the system

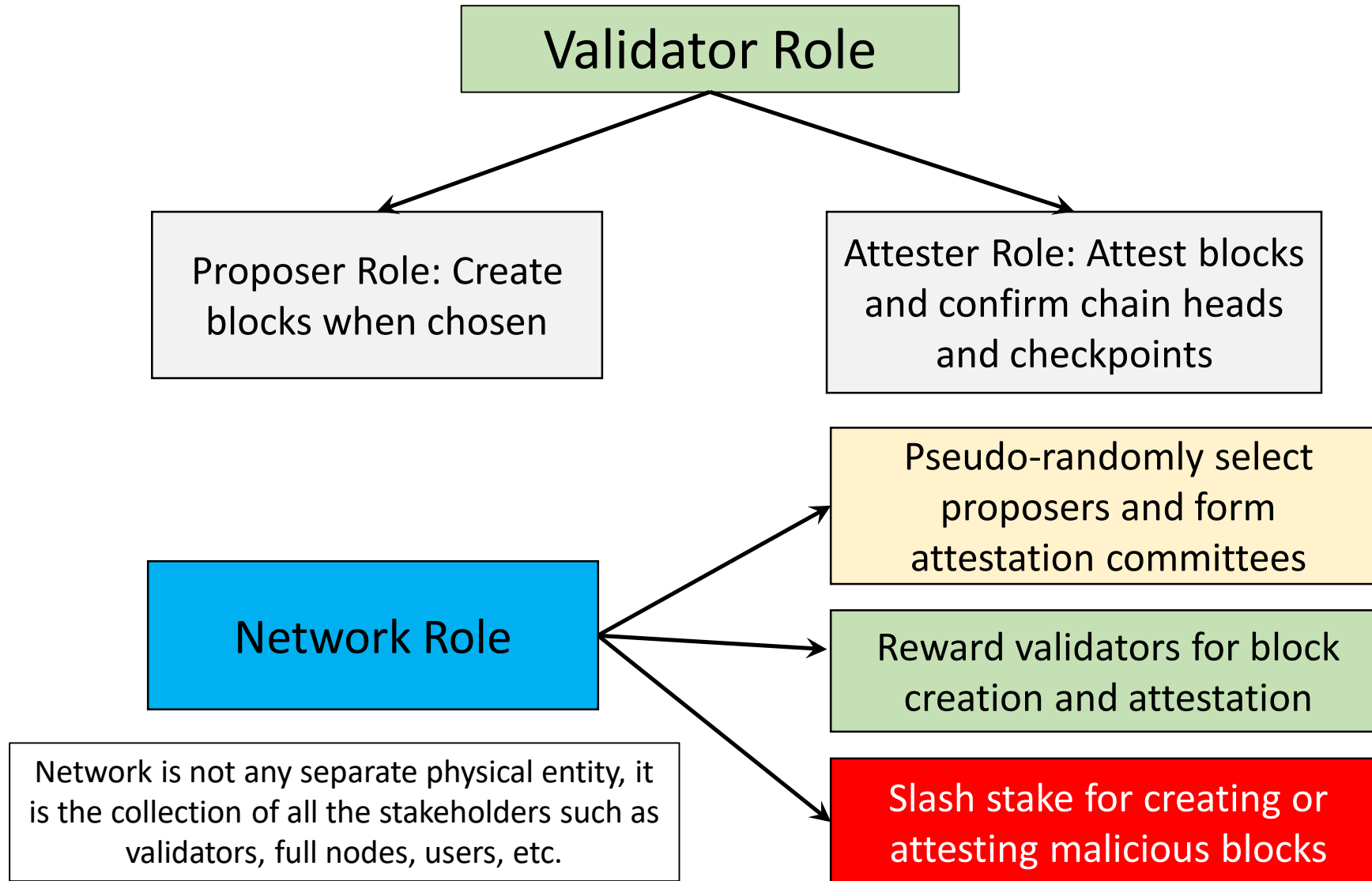
More stake means more chance of creating the next block. For example, if Alice has staked 320 ETH and Bob has only 32 ETH, then Alice can activate 10 validators and has 10 times more probability of creating the next block than Bob

A validator is chosen at random by the network (using RANDAO mechanism) to create new blocks, share them with the network and earn rewards (no intense computational puzzles are solved)



Becoming a validator is a commitment because a validator is expected to maintain sufficient hardware and connectivity to participate in block validation and proposal

Roles of Validator and Network



Transaction Fee

- $1 \text{ ETH} = 10^{18} \text{ wei} = 10^9 \times (10^9 \text{ wei}) = 10^9 \text{ Gwei}$
 - ETH price is high but wei price is extremely small
 - For convenience, gas prices are expressed in Gwei



- A typical ETH transfer costs 21k gas units
- An ERC-20 token transfer requires around 45k to 75k gas units
- Contract deployment costs millions of gas units depending on code complexity
- Ethereum gas prices are variable, so transaction costs can have a significant financial impact

More Transactions More Gas

Consider the following five accounts in a world state

Account	Account Keys							Balance
Alice	a	1	5	1	2	3	4	2.5 ETH
Bob	b	2	1	1	3	5	5	3.5 ETH
Charlie	c	3	7	d	3	3	7	1.2 ETH
Dan	d	4	f	9	3	6	5	1.5 ETH
Eve	e	5	7	d	3	9	7	0.5 ETH

Let's consider the following set of transactions:

1. Alice pays Bob 0.5 ETH,
2. Bob pays Dan 0.2 ETH,
3. Dan pays Alice 0.8 ETH,
4. Charlie pays Eve 0.7 ETH,
5. Eve pays Dan 0.1 ETH

Assuming all the transactions are successfully executed:

1. Alice's Balance: $2.5 - 0.5 + 0.8 = 2.8$ ETH
2. Bob's Balance: $3.5 + 0.5 - 0.2 = 3.8$ ETH
3. Charlie's Balance: $1.2 - 0.7 = 0.5$ ETH
4. Dan's Balance: $1.5 + 0.2 - 0.8 + 0.1 = 1$ ETH
5. Eve's Balance: $0.5 + 0.7 - 0.1 = 1.1$ ETH

On Ethereum, each transaction is posted individually and each pays gas

How can we reduce the fee, while retaining Ethereum security?

Rollup

In all these examples, individual sheets are rolled up into a single unit through compression & aggregation



Can many Ethereum transactions be rolled up like this to reduce transaction fee & achieve scalability without losing Ethereum's Mainnet security guarantees?

Take Transaction Processing off-chain

- Transactions are atomic so if users directly post them on Ethereum they are supposed to be individually processed
- However, if we process the transactions off-chain (like we did in the payment channels), we can find some low-cost and more scalable solution
- We may be tempted to develop a payment channel type off-chain solution, but we must remember:
 - Ethereum supports smart contracts,
 - it has user accounts, and
 - it maintains a world state which is the ultimate source of truth

The idea is to enable ETH or ERC-tokens transfers, while retaining Ethereum security

Lock Funds in a Smart Contract

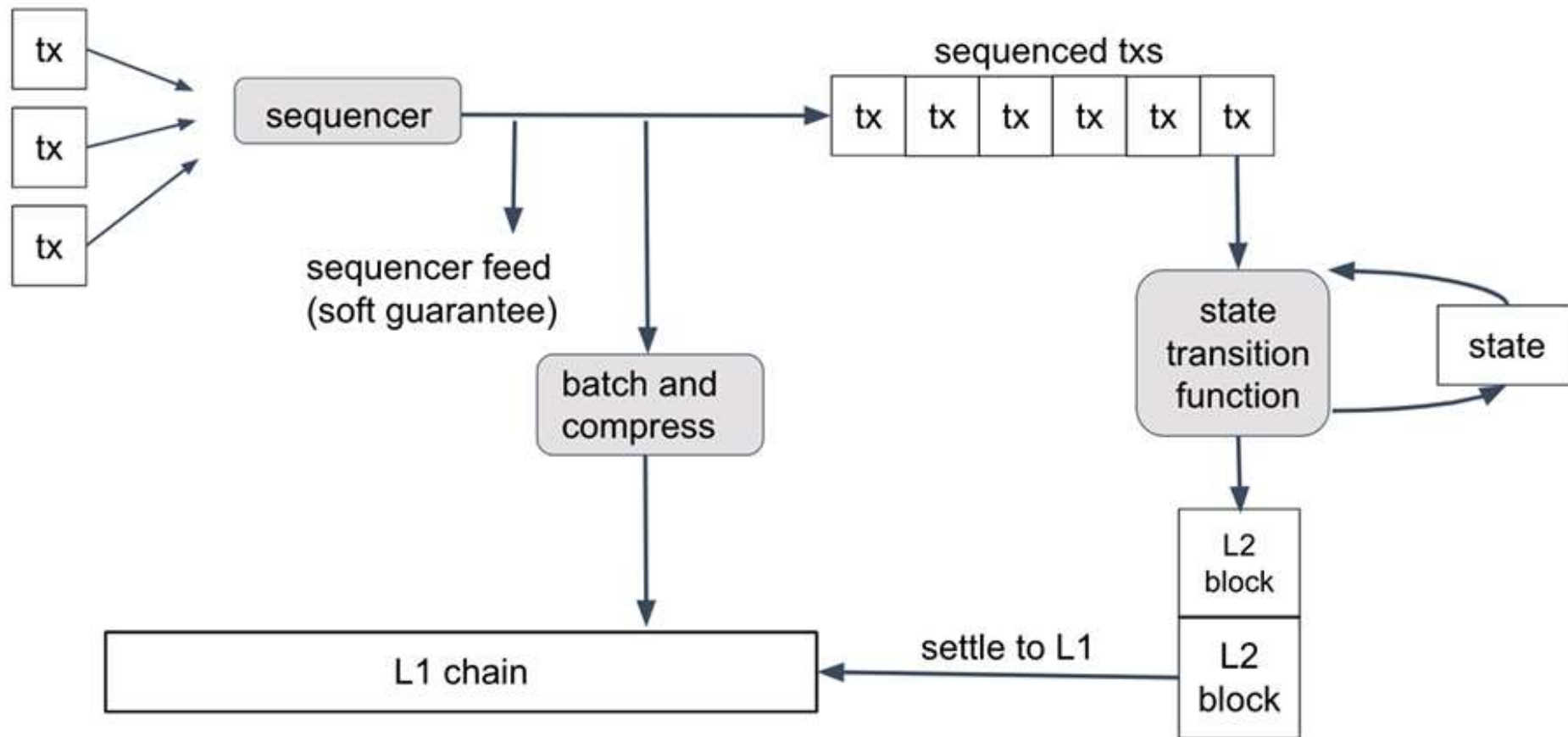
- To develop a solution, the funds (ETH or tokens) must be locked on the Ethereum blockchain
- Any interested user should lock their desired amount of ETH or tokens in a smart contract
 - It does not matter who deploys this smart contract because once it is on blockchain the code governs its behavior independently
 - This step is analogous to posting a funding transaction in a payment channel or a channel factory
- It is also easier to support many users because
 - The smart contract defines all the required rules
 - The user enters the rollup by depositing ETH or tokens in the rollup smart contract
 - The user exits the rollup by withdrawing ETH or tokens from the same smart contract
 - The world state provides a reliable and consistent view of contract's data

Off-Chain Transaction Batching

- Transaction creation, batching and processing is done off-chain
 - A *rollup operator* is responsible for receiving, ordering, executing, and batching transactions off-chain
 - The operator creates a summary of all the state changes that result from these transactions and commits it to Ethereum
- Off-chain transactions are not Ethereum transactions, instead they have a different format
 - The rollup posts the transactions as ‘data’ on Ethereum for security & verification
 - Traditional method for posting data is through the transaction calldata field, which is the data passed along with a transaction for sending messages to other entities or interact with smart contracts
- Calldata method is expensive and costs a significant amount of gas
 - 4 units for each zero byte
 - 16 units for each non-zero byte

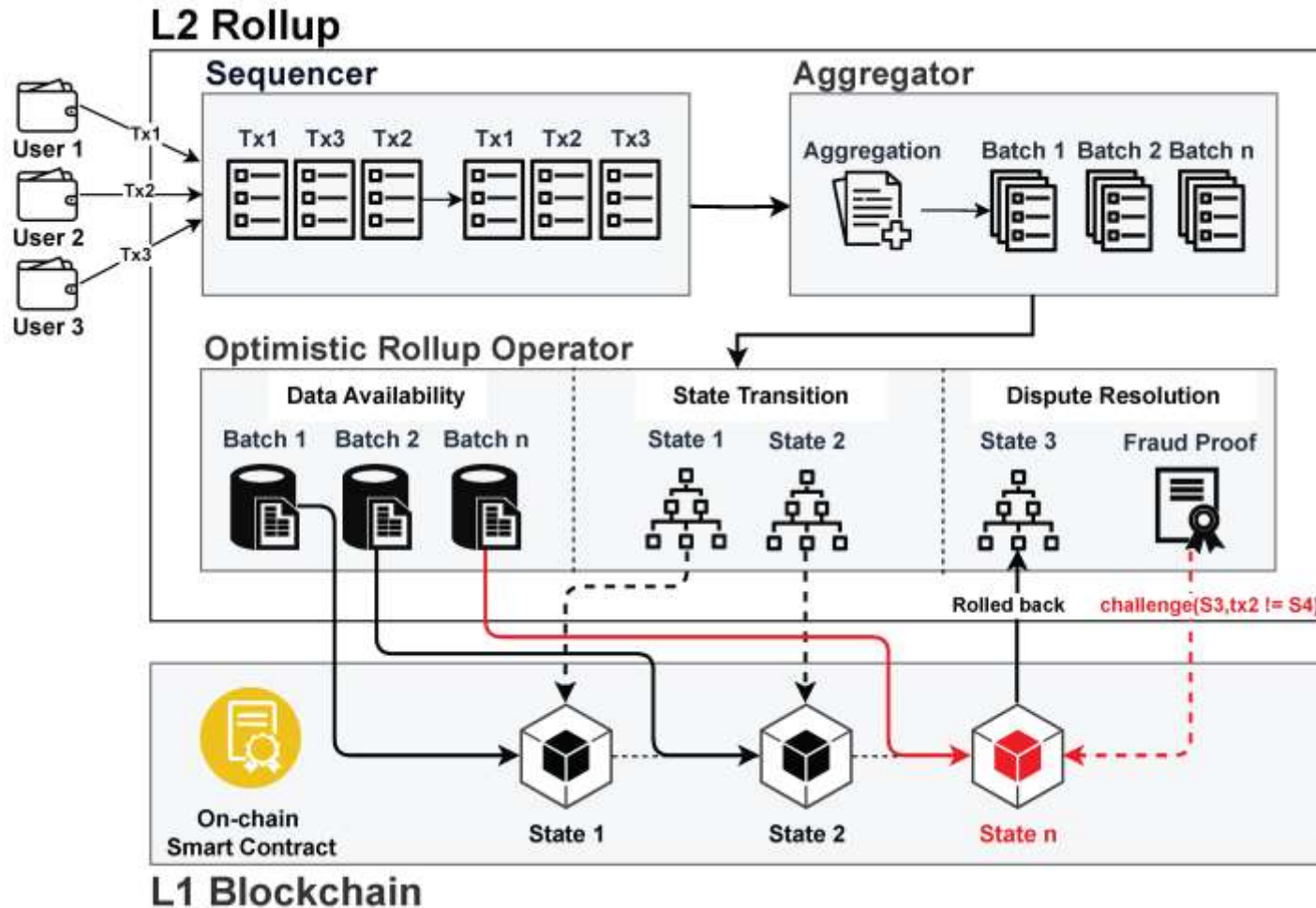
Rollups

Rollups or Ethereum L2 (layer 2) reduce computation on the main Ethereum chain by processing transactions off-chain, offering significant improvements in processing speeds



Optimistic Rollup

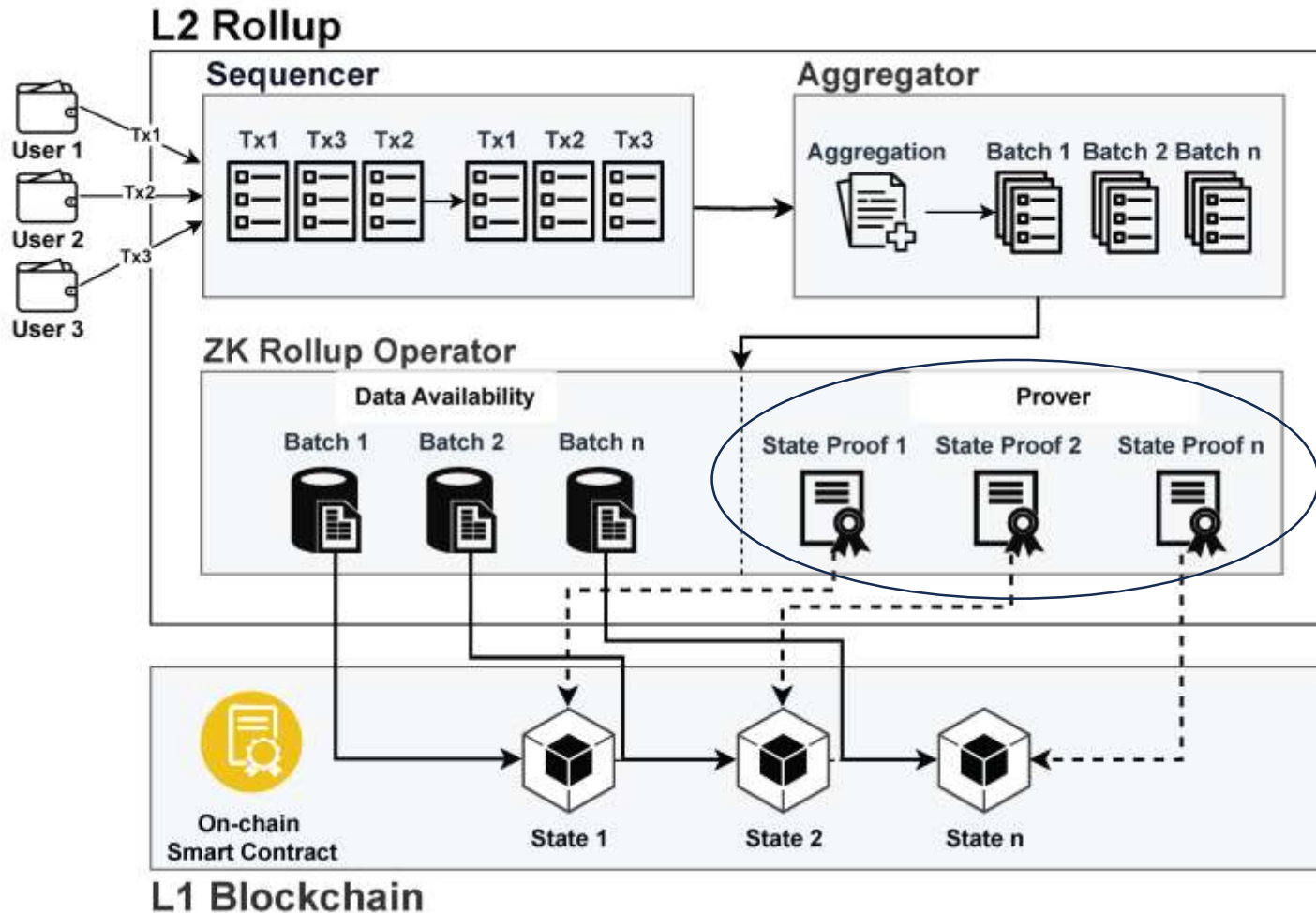
Optimistic rollups assume every transaction to be valid by default and don't publish validity proofs



- Optimistic rollups have a challenge period (usually 7 days) during which anyone can create a dispute
- A fraud proof mechanism is used only to resolve the disputes
- If successful, the disputed states are rolled back and penalties are also enforced (such as, sequencer blacklisting, stake slashing)

ZK-Rollup

ZK-rollups additionally produce cryptographic proofs during off-chain computations



The proofs are also posted on-chain to verify that the computations were performed correctly, and all the state transitions followed the pre-defined rules without revealing the underlying data or the exact computational steps

How Rollups Guarantee Ethereum Security?

- A rollup operator deploys a smart contract on Ethereum, which serves as the main security anchor
- Users lock their funds and tokens in the smart contract
- Users create transactions and submit them to the rollup operator (generally a centralized server)
- The rollup aggregates, batches and compresses transactions and ensures data availability
 - Keeps a local copy of the data
 - Posts data on Ethereum as a data blob – a new and special type of Ethereum transaction introduced to support rollups
- The rollup executes transactions off-chain and computes the state transitions & posts them on Ethereum
- In case of ZK-rollup, the validity proofs are also submitted
- In optimistic rollup, if someone detects an incorrect state update, they can create a dispute that triggers a fraud proof mechanism to ensure correctness
- This process ensures that rollups inherit Ethereum's security, as the Ethereum smart contract enforces finality and correctness

Rollup Ecosystem

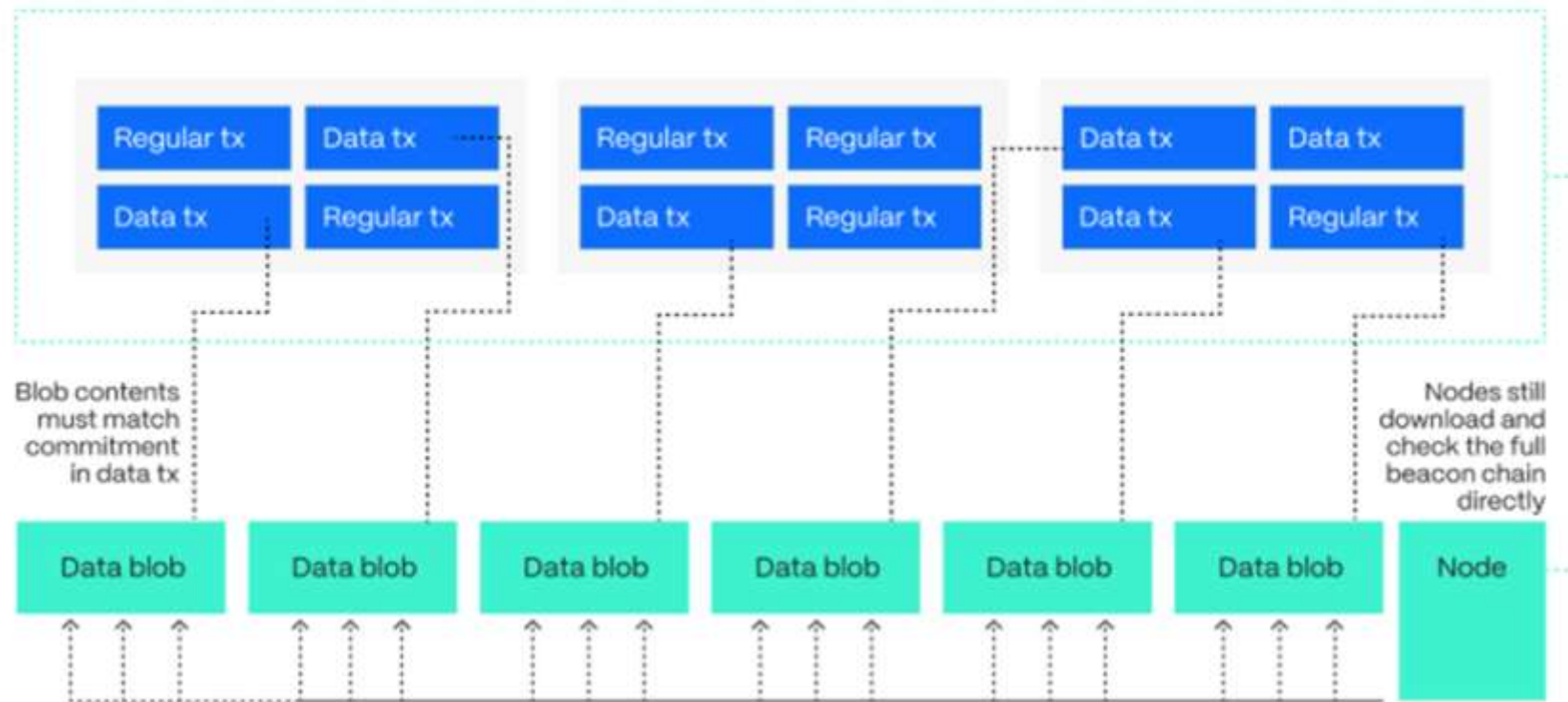
L2 Project	TVL (\$B)	Max TPS	Avg Fees	Daily Active Users (Q1 2025)	Type
Arbitrum	18.0	40,000	\$0.08	487,000	Optimistic Rollup
Base	3.08	2,000	<\$0.01	3.2M (March peak)	Optimistic Rollup
Optimism	2.7	714	\$0.05	750,000+ (Superchain)	Optimistic Rollup
zkSync Era	5.0	20,000	Sub-penny	200,000+	zk-Rollup
Polygon zkEVM	1.7	65,000	<\$0.01	300,000+	zk-Rollup
StarkNet	3.0+	Millions	\$0.02	150,000+	zk-STARK
Mantle	0.2	4,000	80% < ETH	250,000+	Modular L2
Blast	2.68	N/A	<\$0.01	400,000+	Optimistic Rollup

- TVL is the Total value locked in the rollup's Ethereum smart contract
 - Used as a measure of project's adoption
- Average Transaction fee is few cents
 - Optimistic rollups fees are 3x to 8x less than Mainnet
 - ZK-rollups fees are 40x to 100x cheaper than Mainnet due to better data compression
- Ethereum actively supports rollups
 - Considers them essential for its long-term scalability & growth

<https://blog.mexc.com/news/layer-2-revolution-ethereums-scaling-superstars-in-2025-where-speed-meets-billions-in-tvl/>

EIP-4844: Shard Blob Transactions

- To reduce the costs of posting data through the transaction calldata field, Ethereum introduced a new transaction type called proto-danksharding or Type 3 transactions in March 2024
- EIP-4844 transactions are “blob-carrying transactions” or data transactions
- These transactions contain a large amount of data blob that cannot be accessed by EVM execution, but whose commitment can be accessed



<https://hacken.io/discover/eip-4844-explained/>

Regular tx: EIP-1559 format
(Type 2)

Data tx: EIP-4844 format
(Type 3)

Data Blobs



- Each data transaction has a data blob (raw data)
- The maximum size of blob transaction is 128 KiB (131,072 bytes)
- A maximum of 6 data transactions can be attached to a block



This approach provides a maximum increase in data bandwidth of around 768KiB

- Blob gas price is set extremely low (1 wei per gas unit)
- Blob transactions are significantly cheaper than posting an equivalent amount of data via calldata method

Blob Transaction Example

Overview

Hash	0x2a0353844c354ead92379cd6b63b2594634ed86239e10f43fb83963de22a6067
Status	Unfinalized
Block	23811544
Timestamp	3 minutes ago (Nov 16, 2025 4:47:23 PM+05:00)
Position In Block	8
From	0xe2da8ac2e550cd141198a117520d4edc8692ab74
To	0xd5aa0e20e8a6e9b04f080cf8797410fafa9688a
Blob Size	128 KiB (1 blob)
Blob Usage	14.3447265625 KiB 11.2%
Blob Gas Price	0.000000001 Gwei (\$0.000000000000000316741)
Blob Fee	Base: 0.000000000000131072 ether (\$0.00000000041515876352) Max: 0.000000000000131072 ether (\$0.00000000041515876352)
Blob Gas Used	131,072
Blob As Calldata Gas Used	699,056 (5.33 times larger)
Blob As Calldata Gas Fee	0.000699056 ether - 5.33B times more expensive
ETH Price	\$3,167.41

- The 'To' field points to the rollup's L1 smart contract that receives and records the blob commitments
- A single blob always consumes about 131,072 blob-gas units, regardless of how much of the 128 KiB is actually used
- Before blobs (pre-EIP-4844), rollups posted their L2 data as transaction calldata, which was far more expensive and permanently stored on Ethereum, causing long-term chain bloat

<https://blobscan.com/tx/0x2a0353844c354ead92379cd6b63b2594634ed86239e10f43fb83963de22a6067>

Blobs are not Stored Permanently

- The blobs are temporarily stored in the beacon chain by the consensus nodes for around 18 to 21 days before being pruned
- After pruning, the blob data is removed and it is only represented by its KZG commitment (a small cryptographic hash) that was included in its corresponding data transaction
- This design significantly improves gas efficiency by separating blob data from the main execution layer, avoiding the high gas costs of permanently storing large amounts of data on-chain

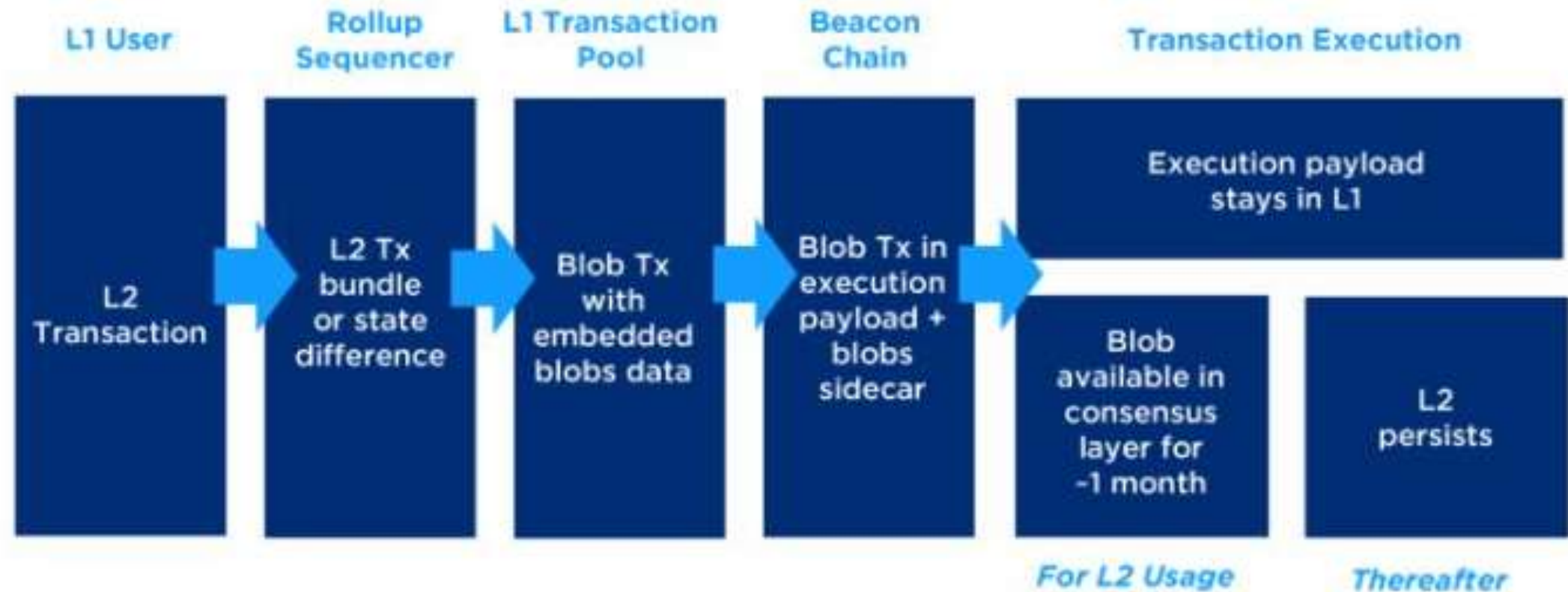
Pruning blob data undermines the idea of permanent blockchain history

However, Ethereum has adopted blob pruning for scalability to keep the blockchain size manageable, while the KZG commitment ensures the integrity and correctness of the pruned data

Blob Transaction Life Cycle

Blob Transaction Life Cycle

Blobs work like sidecars that can be attached to Ethereum blocks

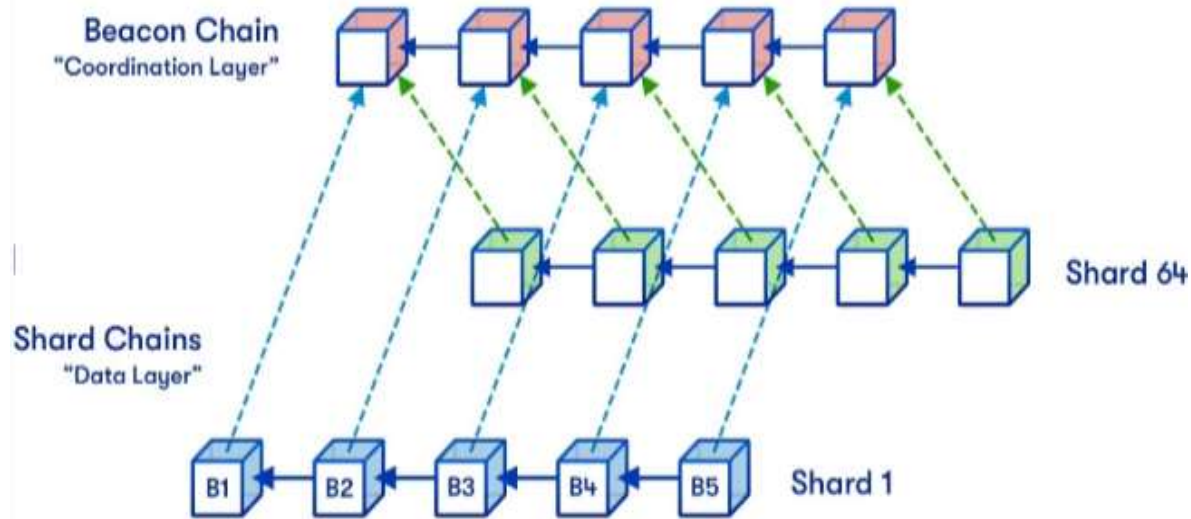


<https://crypto.com/en/research/ethereum-proto-danksharding-dencun>

KZG Commitment

- Blob pruning creates a challenge: once the blob data is deleted, how can nodes verify the integrity of that pruned data safely?
 - Ethereum solves this problem using KZG commitments
- The blob data is divided into bytes which are used as coefficients of a polynomial
 - If there are N bytes, the resulting polynomial is of degree $N-1$
- A secret point s was generated by Ethereum in a KZG ceremony – an event that lasted for several months and concluded in early 2024
 - The idea was that no one should know the full s
- After the ceremony, Ethereum used the secret s to compute and publish a structured reference string (SRS)
- The KZG commitment is computed by taking the dot product of data byte vector with the SRS vector, which is stored in the data transaction
- To prove that a specific coefficient (i.e., a piece of blob data) belongs to the committed polynomial, the prover computes a small KZG proof using the same SRS
- The verifier then checks a pairing equation that confirms the coefficient is correct without needing the blob itself

From Sharding to DankSharding



The idea of sharding was to divide Ethereum blockchain into 64 mini blockchains called shards

- Each shard chain was supposed to process its own transactions and store its own state (with possible cross-shard communication)
- The consensus layer was supposed to be shared among all the shards as they all rely on the same PoS validator set

- Ethereum has since replaced chain sharding with so-called danksharding (also called data sharding)
- This new approach was informally called “Dankrad’s sharding design” that quickly shortened to “Danksharding”

Shard chains are *no longer* part of the Ethereum roadmap

DankSharding

- Danksharding is the full realization of the rollup scaling that began with Proto-Danksharding or EIP-4844
- The blobs attached to blocks will be increased from 6 in Proto-Danksharding to 64 in full Danksharding
- The rest of the changes required are all updates to the way consensus clients operate to enable them to handle the new large blobs
- Danksharding once fully implemented aims to make transactions on Layer 2 as cheap as possible for users and should scale Ethereum to >100,000 transactions per second

Data Sharding

- With full danksharding the block size will become 8 MB
- Ethereum produces a block every 12 s
 - 5 blocks per minute
 - 300 blocks per hour
 - 7200 blocks per day
- Without danksharding blockchain growth
 - 40 MB per minute
 - 2.4 GB per hour
 - 57.6 GB per day
 - 1.72 TB per month
- Very quickly becomes unmanageable – explains the need for data sharding instead of chain sharding

Entities Required for Automation

Suppose, Alice wants to set up a trust fund to pay Bob \$100 at the start of each month for the next 12 months. She writes an Ethereum smart contract (encode conditions) and lock ETH into it

Smart contract cannot initiate transactions, therefore, some entity with EOA must create transactions at the start of each month

Who performs this job?

Smart contract needs the date information; therefore, some entity must provide the current date reliably

Who performs this job?

Smart contract needs to do the ETH/USD conversion; therefore, some entity must provide the conversion rate

Who performs this job?

Smart contract has the ability to check whether the required conditions (according to the code) are met before initiating the funds transfer

Some other entities are needed to automate smart contract execution

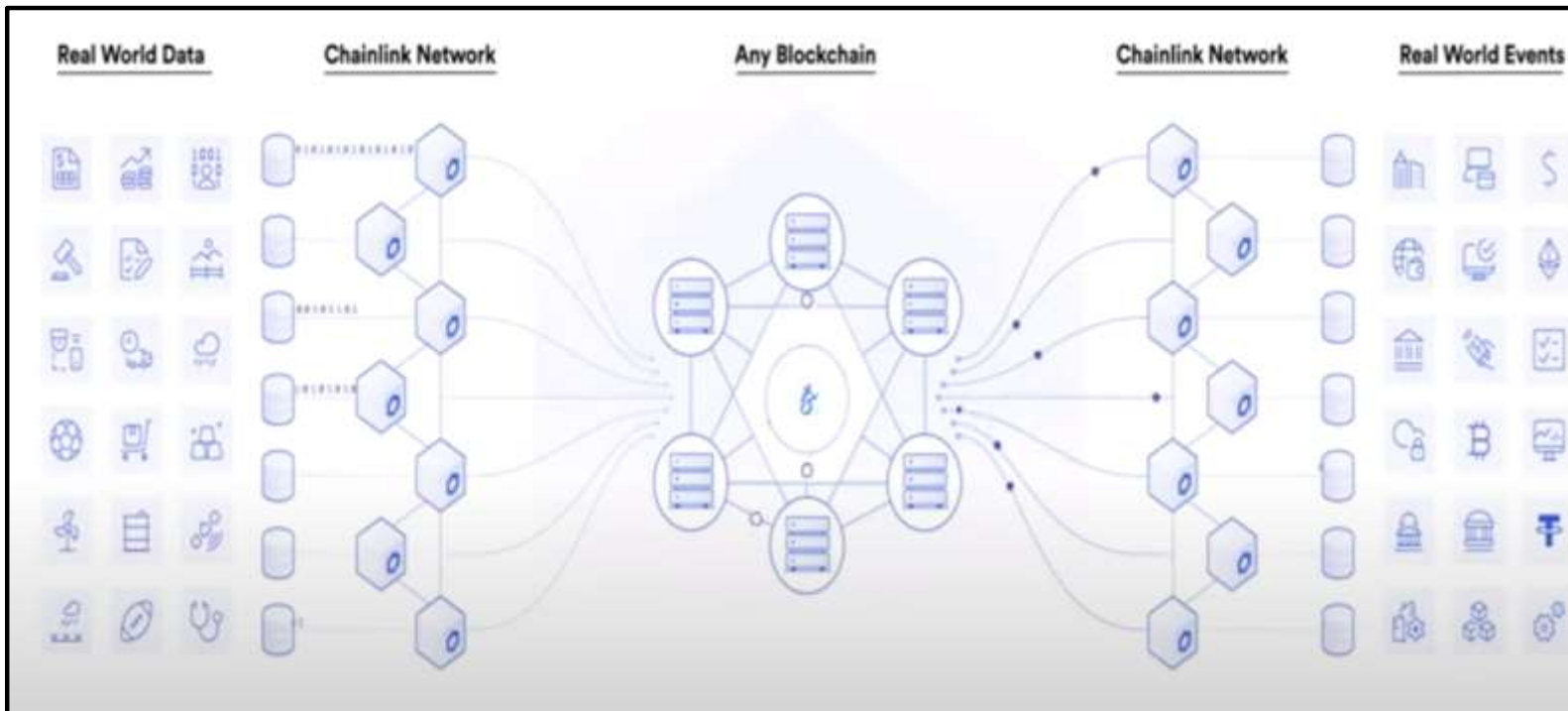
Keepers & Oracles

Keepers: These are external entities that provide services to trigger state updates

- Keepers, are generally financially incentivized to trigger such state updates

Oracles: These are external entities for reliably providing off-chain data to the smart contracts

- Decentralized oracle: multiple nodes maintained by a separate network to provide data feeds (e.g., Chainlink); rely on incentives for accurate and honest reporting



Oracles also post the queried data on the blockchain in the form of transactions. Why?

- Blockchain is deterministic and therefore nodes at any point in time should be able to validate the transactions using the stored data feed transaction

Keepers & Oracles for Full Automation

Suppose, Alice wants to set up a trust fund to pay Bob \$100 at the start of each month for the next 12 months. She can write an Ethereum smart contract (encode conditions) and lock funds into it

Smart contract cannot initiate transactions, therefore, some entity with EOA must create transactions at the start of each month

Keepers perform this job

Smart contract needs the date information; therefore, some entity must provide the current date reliably

Oracles perform this job

Smart contract needs to do the ETH/USD conversion; therefore, some entity must provide the conversion rate

Oracles perform this job

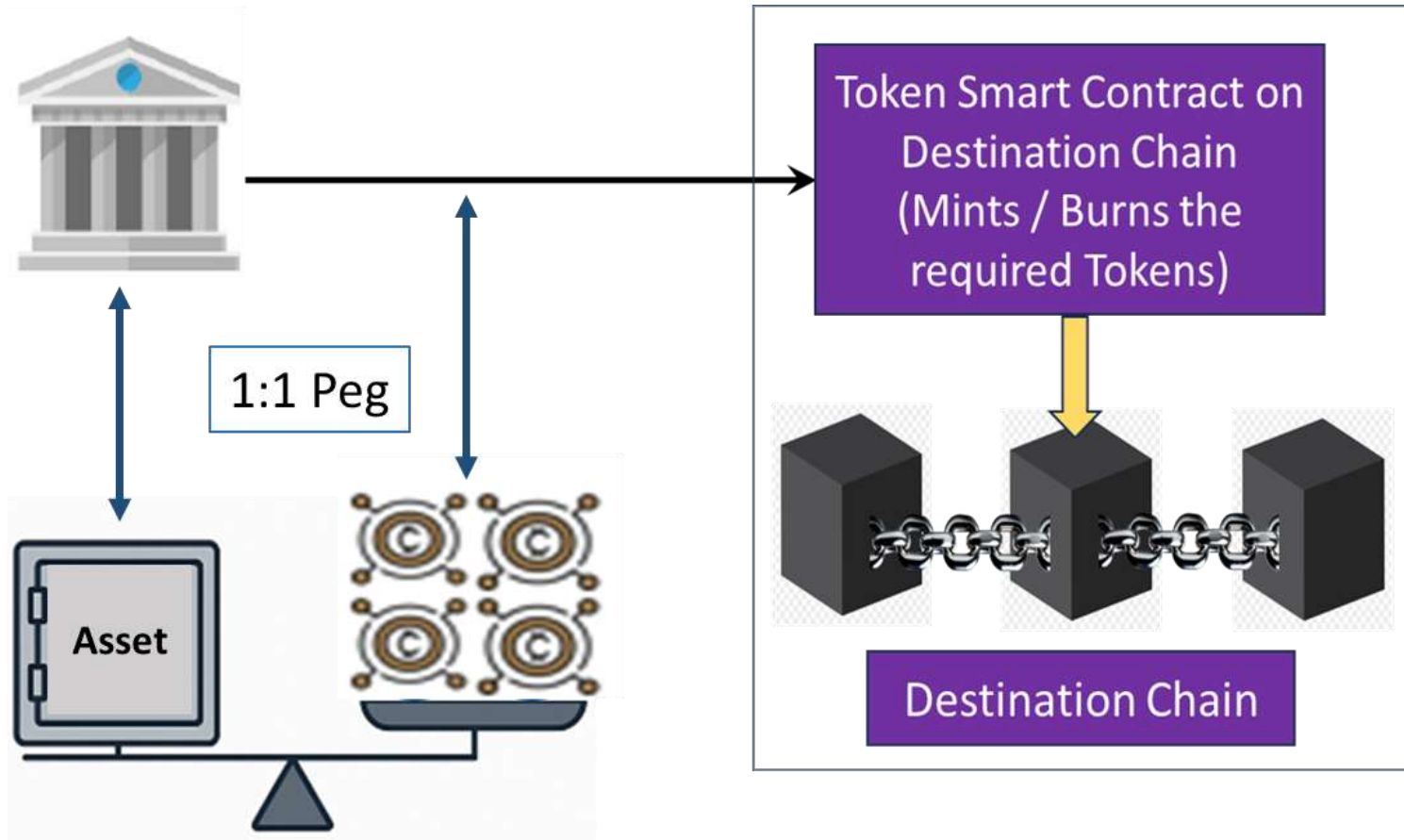
Smart contract has the ability to check whether the required conditions (according to the code) are met before initiating the funds transfer

Smart contract performs this job

Keepers and oracles are essential to achieve full automation

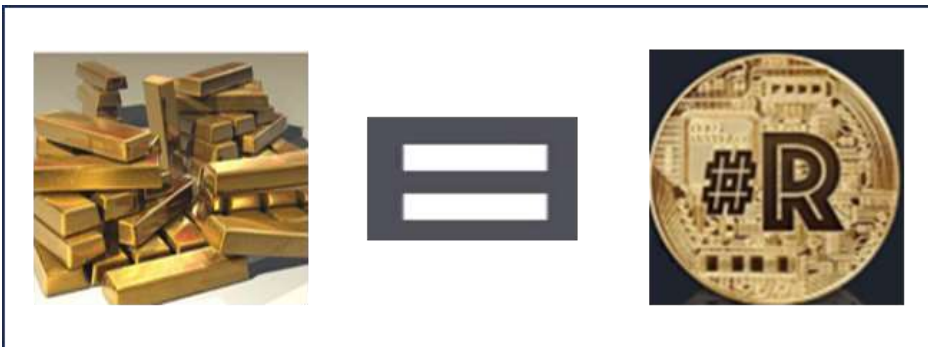
Value Transfer from one World to Another

- Moving the value of any non-native asset from one world to another
- Moving the value of any non-programmable asset to a programmable blockchain

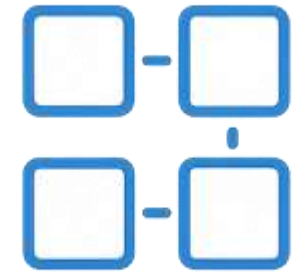


- Stable coins
- Real world Asset (RWA) tokenization
- Wrapped BTC
- Cross-chain bridges

Programmable Value



AUTOMATION



COMPOSABLE



EFFICIENT



**TRUST
MINIMIZATION**